

A.A. 2026-2027

Reti e Programmazione Distribuita

Indice

0 Note sul corso 3

I Fondamenti 4

1 Introduzione alle reti di calcolatori 5
Rete di calcolatori 5
Internet 5
Componenti di una rete 5
Comunicazione dei dati 6
Topologie di rete 7
Categorie di reti 8
ISP e accesso a Internet 8

2 Internet protocolli e livelli 10
Vista “nuts and bolts” di Internet 10
Protocolli e standard 10
Il modello a livelli 11
Servizi di connessione e affidabilità 11
Il modello ISO/OSI 12
1. Livello fisico 12
2. Livello data link 12
3. Livello di rete 13
4. Livello di trasporto 13
5. Livello di sessione 13
6. Livello di presentazione 14
7. Livello applicazione 14
Incapsulamento 15
Critiche e confronto TCP/IP vs OSI 15

II Application Layer 16

3 Application Layer 17
Applicazioni di rete 17

Architetture delle applicazioni 17
Comunicazione tra processi 18
Servizi di trasporto 18
Protocolli applicativi 19
FTP e FTPS 28

4 Application Layer HTTP 21
Web e HTTP 21
URL 21
Il protocollo HTTP 22
Stateless 22
Connessioni persistenti e non persistenti 23
Connessione non persistente 23
RTT e tempi di risposta 24
Connessione persistente 24
Confronto persistente vs non persistente 25

5 HTTP messaggi cookie e web caching 26
HTTP riepilogo 26
Formato dei messaggi 26
Messaggio di richiesta 26
Messaggio di risposta 27
Cookie 29
Web caching 30
Oltre il caching 31

6 Application Layer posta e-mail e P2P 32
Posta elettronica e SMTP 32
Accesso alla posta 33
POP3 33
IMAP 33
Accesso via HTTP 33
Peer-to-Peer 34
BitTorrent 34
Il tracker 34
Strategie del client 34

7 Application Layer DNS 36
Identificazione degli host 36
Resolver 36
Il name space DNS 37
Resource record 37
Tipi di record principali: 38
Name server e zone 39
Name resolution 39
Il resolver locale e i servizi aggiuntivi 40
Strumenti di laboratorio 41
`nslookup` 41

`dig` (Domain Information Groper) 42

8 Lezione 8 Programmazione di rete 44

Dal livello di applicazione alle socket 44
Socket come interfaccia tra processi e trasporto 45
Connectionless e connection-oriented 45
La funzione socket 45
Strutture degli indirizzi 46
`sockaddr_in` 46
Rappresentazione dei numeri e byte order 46
`bind` e assegnazione dell'indirizzo 47
Binding implicito 47
UDP 47
Schema di comunicazione 47
`connect` su socket UDP 48
Flag di `recvfrom` e `sendto` 48
Datagram non attesi 48
Timeout di ricezione 49
Comunicazione su localhost 49
Server UDP multi-client 49
Buffer delle socket 49
Invio UDP 49
Ricezione UDP 50
TCP 50
Connessione e handshake 50
Identificazione delle connessioni 50
Chiusura della connessione 51
Opzioni dei socket 51
Flag di `send` e `recv` 51
Modalità non bloccante del descrittore 52
Buffer e comportamenti blocking 52
Letture e scritture robuste 52
Server concorrenti 53
Server sequenziale 53
Server multiprocesso 53
Server multithread 54
Server multiplexing con `select` 54

Capitolo 0

Note sul corso

I seguenti appunti sono stati presi durante le lezioni del corso di **Reti e Programmazione Distribuita**, a.a. 2026/2027.

Gli appunti sono organizzati in cinque parti che seguono i livelli dello stack TCP/IP:

1. **Introduzione alle reti** (L01–L02): fondamenti, storia di Internet, modelli a livelli, OSI vs TCP/IP.
2. **Application Layer** (L03–L09): HTTP, e-mail, DNS, P2P, socket e programmazione di rete.
3. **Transport Layer** (L10–L14): porte e UDP, trasferimento affidabile (GBN, SR), TCP, controllo di flusso e congestione.
4. **Network Layer** (L15–L21): forwarding e router, indirizzamento IPv4/IPv6, DHCP, NAT, routing Distance-Vector e Link-State.
5. **Link Layer e Sicurezza** (L22–L26): framing e gestione degli errori, protocolli MAC, switch e ARP, crittografia, SSL/TLS, firewall e IDS.

Nota: gli appunti sono redatti a partire dalle slide ufficiali del corso e dalle loro trascrizioni.

Parte I

Fondamenti

Capitolo 1

Introduzione alle reti di calcolatori

Rete di calcolatori

Definition

Rete di calcolatori

Insieme di dispositivi di calcolo autonomi ma interconnessi: un numero elevato di computer separati che, grazie ai collegamenti, possono **scambiarsi informazioni** per svolgere un lavoro comune.

Gli usi principali delle reti di calcolatori sono:

- **Accesso all'informazione**, tipicamente secondo il modello client-server o peer-to-peer.
- **Comunicazione persona-persona** ed e-commerce.
- **Intrattenimento** e *Internet of Things* (quasi qualsiasi dispositivo può connettersi).

Nel **modello client-server** un client richiede esplicitamente informazioni a un server che le ospita; il processo client invia una richiesta in rete e attende la risposta. In un sistema **peer-to-peer** non esistono client e server fissi.

Internet

Definition

Internet

Rete di reti: la più grande rete di calcolatori, tecnicamente un enorme **WAN** che collega sotto-reti e dispositivi su nazioni e continenti.

Cronologia essenziale:

- **1950 ca.**: l'idea di una rete "universale" viene teorizzata (J.C.R. Licklider).
- **1969**: l'ARPA (U.S. Dept. of Defense) finanzia **ARPANET**, inizialmente una rete chiusa di università e centri di ricerca (primi nodi: SRI, UCLA, UCSB, Utah).
- **1974**: R. Kahn e V. Cerf progettano **TCP** e **IP**, i due protocolli della *Internet protocol suite*.
- **1986**: la NSF finanzia **NSFNET**, rete basata su TCP/IP; essa e TCP/IP avviano l'idea di *network of networks*, ma entrambe le reti hanno restrizioni commerciali.
- **1990**: ARPANET viene dismessa; **1995**: NSFNET viene dismessa, rimuovendo le ultime restrizioni al traffico commerciale.

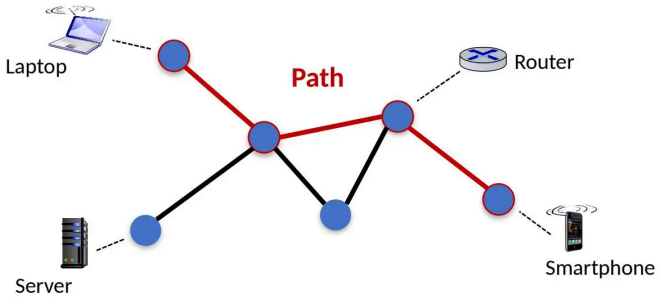
Componenti di una rete

La rete distingue l'infrastruttura dai sistemi finali:

- **Dispositivi di rete** (*network devices*): router, switch, access point, hub, insieme ai **link** (canali di comunicazione), formano l'infrastruttura che permette ai host di connettersi.
- **Host**: i dispositivi su cui girano applicazioni o programmi (PC, laptop, smartphone, server).

Le reti condividono la terminologia della teoria dei grafi:

- I dispositivi connessi sono i **nodi**, le connessioni sono i **link** (o canali).
- Una **sequenza** di nodi e link è un **cammino** (*path*).
- I **host** (o sistemi finali) sono i nodi che producono o usano i **servizi**; idealmente sono le foglie della rete, mentre i nodi intermedi sono dispositivi di instradamento.

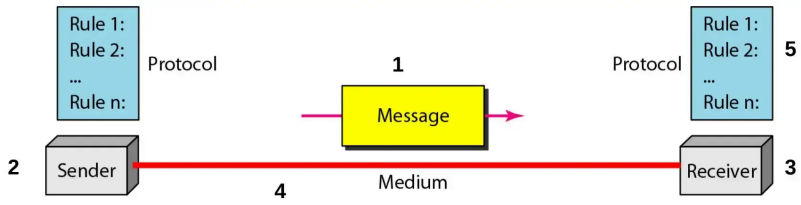


Comunicazione dei dati

Definition

Comunicazione dei dati

Scambio che coinvolge cinque componenti: **messaggio** (i dati da trasmettere), **mittente**, **destinatario**, **mezzo** (canale tra mittente e destinatario) e **protocollo** (regole di comunicazione note a entrambi).



I dati si rappresentano in forme diverse: testo, numeri, immagini, audio, video. Il **flusso** dipende dal tipo e dallo scopo della comunicazione:

- **Simplex**: monodirezionale.
- **Half-duplex**: bidirezionale, ma a turni alterni.
- **Full-duplex**: bidirezionale simultaneo.

Tre grandezze misurano il flusso, in bit/s (o byte/s):

- **Transmission rate**: quantità *massima* di informazione trasmissibile da un *dispositivo*.
- **Banda (bandwidth)**: quantità *massima* trasmissibile da un *cammino* (link e nodi).
- **Throughput**: quantità *effettiva* (istantanea) trasmessa su un cammino.

Le connessioni tra host possono essere di due tipi:

- **Point-to-point**: link dedicato tra due soli dispositivi (cablato o wireless).
- **Multipoint (broadcast)**: più di due dispositivi condividono lo stesso link.

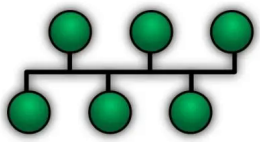
Topologie di rete

Definition

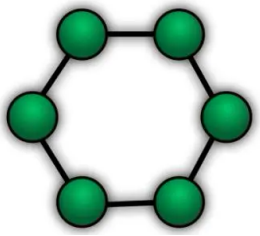
Topologia di rete

Disposizione degli elementi (nodi, link) di una rete di comunicazione.

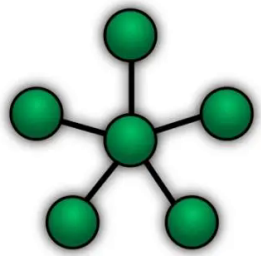
- **Bus**: gli host sono collegati a un cavo dorsale centrale; possibili **collisioni**; semplice ed economica, adatta a reti piccole, ma la dorsale è un punto singolo di guasto. Ha 1 link fisico duplex (o 1 dorsale e `n` link).



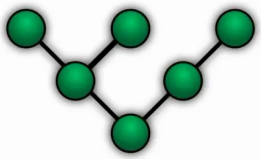
- **Ring**: ogni host è connesso point-to-point a esattamente altri due; il segnale è **inoltrato** lungo l'anello fino alla destinazione; semplice ed economica, con prestazioni migliori del bus, ma aggiungere nodi è più difficile e un guasto di un nodo può compromettere la rete; ha `n` link duplex.



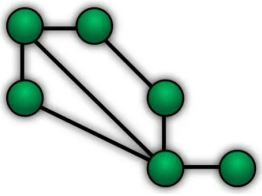
- **Star**: gli host sono collegati a un controllore centrale (hub, switch o router) che reindirizza i messaggi, senza link diretti tra host; economica, semplice, robusta e scalabile, ma il controllore deve essere raggiungibile da tutti ed è un punto singolo di guasto; ha `n` link duplex.



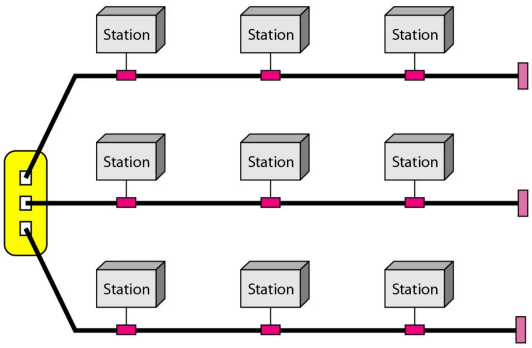
- **Tree**: più topologie a stella integrate, tipicamente tramite un cavo bus; versatile, scalabile, robusta e ben supportata, ma difficile da configurare ed eredita la debolezza del bus.



- **Mesh**: host collegati point-to-point in modo non gerarchico; nella **full mesh** tutti i nodi sono tra loro connessi ($n(n-1)/2$ link duplex), nella **partial mesh** solo alcuni; poco traffico, robusta, sicura, con link dedicati, ma difficilmente scalabile e costosa.



- **Hybrid**: topologie diverse possono essere **mixate** in una rete ibrida.



Categorie di reti

Le reti si classificano per dimensione, numero di host e banda:

- **PAN** (*Personal Area Network*): dispositivi nel raggio di una persona (es. Bluetooth).
- **LAN/WLAN** (*Local/Wireless Area Network*): rete locale cablata (es. Ethernet switchata) o wireless (es. 802.11).
- **MAN** (*Metropolitan Area Network*): copertura metropolitana (es. distribuzione di TV e Internet tramite *cable head-end*).
- **WAN** (*Wide Area Network*): copertura di nazioni/continenti, tipicamente molto complessa ed eterogenea; la complessità della topologia cresce con la dimensione della rete.

La **rete casalinga** (home LAN) ospita una gamma ampia e diversificata di dispositivi connessi a Internet (IoT): deve essere facile da installare, sicura e affidabile, con interfacce compatibili tra tutti i prodotti.

ISP e accesso a Internet

Definition

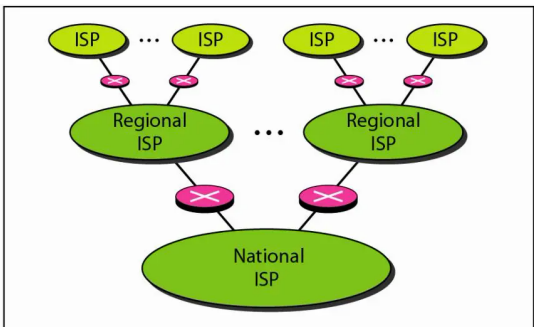
ISP

Internet Service Provider: organizzazione (commerciale, comunitaria, non-profit, privata) che fornisce servizi di accesso, uso e partecipazione a Internet.

- Le connessioni private avvengono tramite modem/ONT collegato alla rete telefonica; le aziende (medie/grandi) possono avere un collegamento diretto/dedicato all'ISP.
- Differenti ISP si scambiano dati tramite **NAP** (*Network Access Point*) o **IXP** (*Internet Exchange Point*), punti di incontro tra più ISP spesso non gestiti dagli ISP coinvolti.

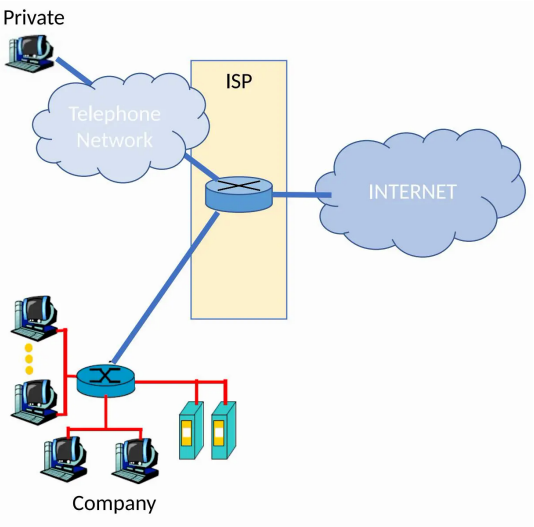
La rete di un ISP è gerarchica:

- **PoP** (*Point of Presence*): gruppo di uno o più router con cui l'ISP raggiunge i clienti.
- **Access ISP** per aree locali, **regional ISP** per aree più ampie, **national ISP** a livello di nazione.



Tecnologie di connessione privata (in ordine di banda):

- Modem analogico (56 kbps), **ISDN** (128 kbps), **ADSL** (1–20 Mbps), doppino in rame (10–100 Mbps), fibra ottica (50 Mbps–40 Gbps, tramite **ONT** senza (de)modulazione).



Capitolo 2

Internet protocolli e livelli

Vista "nuts and bolts" di Internet

Dalla prospettiva dell'implementazione, Internet è composta da:

- **Host** (o *end systems*): miliardi di dispositivi connessi che eseguono le applicazioni di rete al "bordo" di Internet.
- **Packet switch**: dispositivi che inoltrano pacchetti (blocchi di dati), principalmente router e switch.
- **Link di comunicazione**: fibra, rame, radio, satellite; il tasso di trasmissione è la **banda**.
- **Reti**: insiemi di dispositivi, router e link gestiti da un'organizzazione.

Info

Internet

Internet è una **rete di reti** di ISP interconnessi, in cui i protocolli sono ovunque: governano l'invio e la ricezione dei messaggi (es. HTTP, streaming video, TCP, IP, WiFi, 4G, Ethernet).

Dalla prospettiva dei servizi, Internet è un'**infrastruttura** che offre servizi alle applicazioni (Web, e-mail, e-commerce, social, …) e fornisce una **interfaccia di programmazione** alle applicazioni distribuite: "agganci" per connettersi al servizio di trasporto, con opzioni di servizio analoghe al servizio postale.

Protocolli e standard

Definition

Protocollo di rete

Insieme di regole che definisce il *formato*, l'*ordine* dei messaggi scambiati tra le entità di rete e le *azioni* intraprese alla trasmissione e alla ricezione dei messaggi.

I protocolli umani (domande, presentazioni) e i protocolli di rete condividono la stessa struttura: messaggi specifici inviati e azioni specifiche compiute alla ricezione di un messaggio o di un altro evento; nei protocolli di rete, però, le entità sono computer.

Gli standard di Internet sono definiti da una comunità di esperti:

- **IETF** (*Internet Engineering Task Force*): organizzata in **working group** aperti, ciascuno focalizzato su un aspetto specifico di Internet.
- **RFC** (*Request For Comments*): documenti numerati prodotti dai working group, con descrizione e definizione di protocolli, concetti e metodi alla base di uno standard (es. IPv4 è definito nell'RFC 791, 1981).

Il modello a livelli

Definition

Layering

Approccio architetturale alla comunicazione: il sistema è diviso in livelli (*divide et impera*), ciascuno un livello di astrazione che **usa i servizi del livello inferiore e offre servizi al livello superiore** attraverso interfacce.

Il modello a livelli serve a domare la complessità della rete (host, router, link eterogenei, applicazioni, protocolli, hardware e software):

- La struttura esplicita permette di **identificare** e mettere in relazione i pezzi del sistema complesso.
- La **modularizzazione** facilita manutenzione e aggiornamento: cambiare l'implementazione del servizio di un livello è trasparente al resto del sistema.
- **Contro**: il layering ha un costo in efficienza ("*layering considered harmful*").

Quando due dispositivi comunicano, **ogni livello del mittente comunica con lo stesso livello del destinatario** mediante un protocollo specifico.

Servizi di connessione e affidabilità

Definition

Servizio connection-oriented

Servizio orientato alla connessione modellato sul sistema telefonico: l'utente prima **stabilisce** la connessione, la usa, poi la rilascia; può negoziare i parametri da usare.

Definition

Servizio connectionless

Servizio senza connessione modellato sul sistema postale: i pacchetti vengono inviati indipendentemente, senza setup.

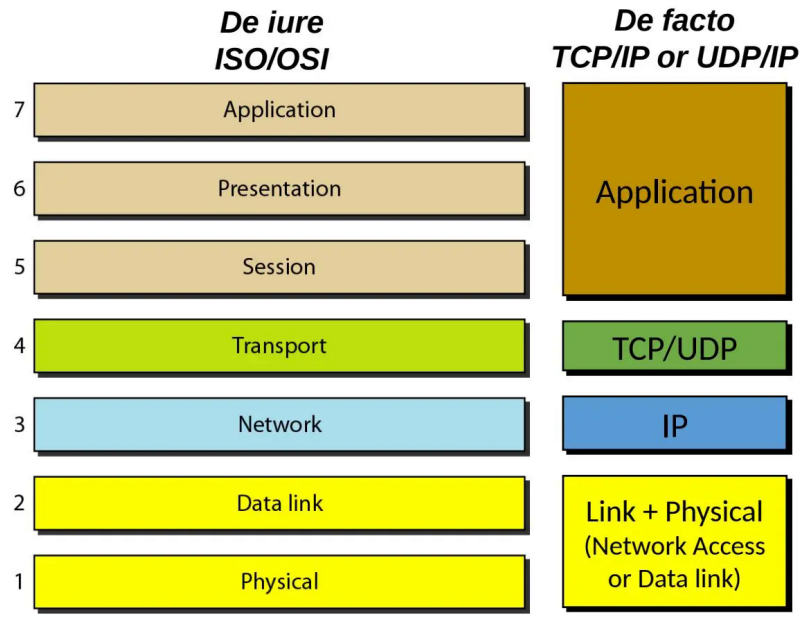
Alcune nozioni collegate:

- **Pacchetto** (*packet*): un messaggio al livello di rete.
- **Store-and-forward switching**: i nodi intermedi ricevono il messaggio per intero prima di inoltrarlo al nodo successivo.
- **Cut-through switching**: la trasmissione presso un nodo inizia prima che il messaggio sia ricevuto completamente.
- **Servizio datagram**: servizio connectionless non affidabile (senza conferme).
- L'**affidabilità** caratterizza sia i servizi orientati sia quelli non orientati alla connessione.

I sei servizi tipici orientati e non orientati alla connessione sono i seguenti.

Tipo	Servizio	Esempio
Connection-oriented	Reliable message stream	Sequenza di pagine
Connection-oriented	Reliable byte stream	Download di un film
Connection-oriented	Unreliable connection	Voice over IP
Connectionless	Unreliable datagram	Posta elettronica di massa
Connectionless	Acknowledged datagram	Messaggistica testuale
Connectionless	Request-reply	Interrogazione di database

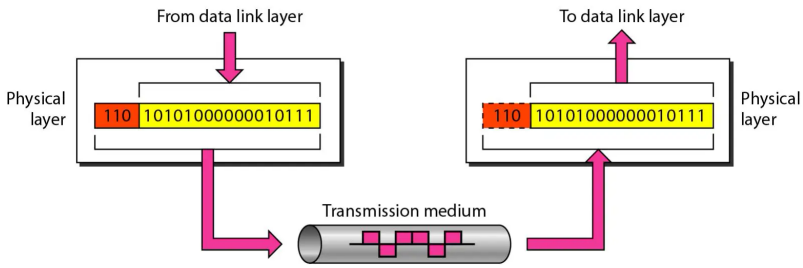
Il modello ISO/OSI



Negli anni '80 l'**ISO** (*International Standards Organization*) ha definito il modello a livelli **OSI** (*Open System Interconnection*), con 7 livelli:

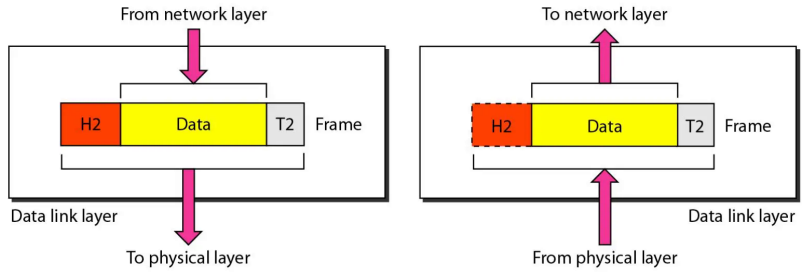
1. Livello fisico

Il **livello fisico** è responsabile del movimento dei singoli bit da un hop (nodo) al successivo. Regola le caratteristiche fisiche di interfacce e mezzi (connettori, cavi, segnali elettrici), la configurazione della linea (punto-punto o multipunto), la topologia fisica (mesh, star, ring o bus), la modalità di trasmissione (simplex, half-duplex o duplex), la rappresentazione dei bit, la velocità di trasmissione (*data rate*) e la sincronizzazione dei bit.



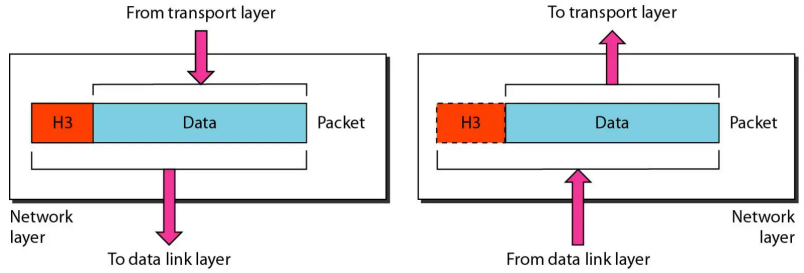
2. Livello data link

Il **livello data link** è responsabile del movimento dei frame da un hop al successivo (segmento). Lavora su frame, porzioni di dati tipicamente di poche centinaia di byte, e si occupa di controllo di flusso ed errori (tramite sequenze di controllo del frame) e di controllo di accesso.



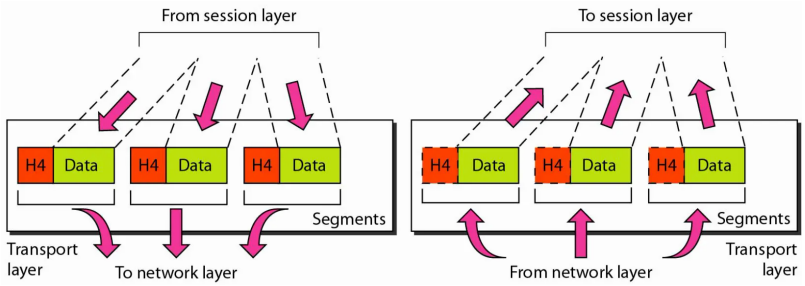
3. Livello di rete

Il **livello di rete** è responsabile della consegna dei singoli pacchetti dall'host sorgente all'host destinazione (cammino). Lavora su pacchetti, tipicamente più grandi e complessi dei frame, e gestisce la consegna da sorgente a destinazione, l'indirizzamento logico e il routing tramite tabelle di routing.



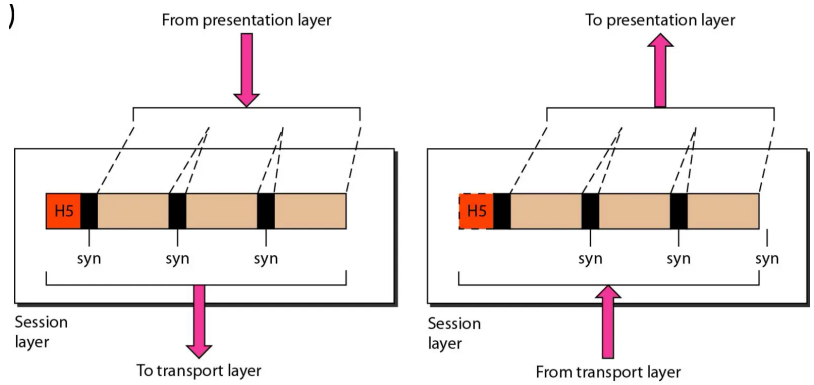
4. Livello di trasporto

Il **livello di trasporto** è responsabile della consegna di un messaggio da un estremo all'altro, così che il messaggio arrivi al programma giusto. Si occupa di consegna end-to-end, controllo della connessione (orientato o senza connessione), segmentazione/riassemblaggio da e verso i pacchetti del livello 3, indirizzamento delle porte e controllo di flusso ed errori.



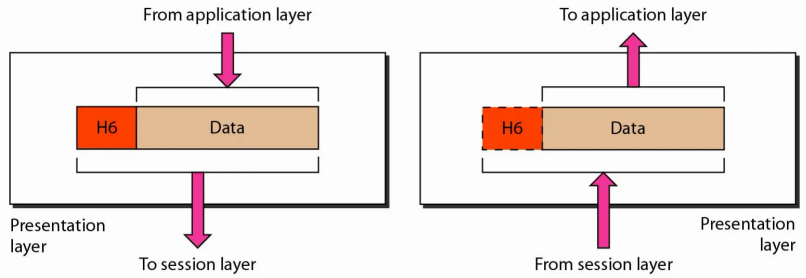
5. Livello di sessione

Il **livello di sessione** è responsabile del controllo del dialogo e della sincronizzazione richiesta/risposta: stabilisce, mantiene e sincronizza l'interazione tra i sistemi che comunicano (sessione di comunicazione), gestisce il dialogo e prevede punti di controllo (*checkpoint*) per la sincronizzazione.



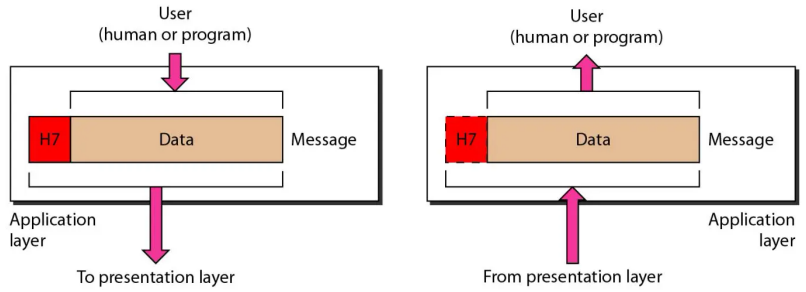
6. Livello di presentazione

Il **livello di presentazione** è responsabile della rappresentazione dei dati: si occupa di traduzione (ad esempio da codifica EBCDIC o ASCII a testo), cifratura e decifratura, e compressione.



7. Livello applicazione

Il **livello applicazione** è responsabile di fornire servizi all'utente: terminale virtuale di rete (accesso remoto), trasferimento e accesso a file, servizi di posta e accesso al World Wide Web.



Non sempre tutti i livelli sono implementati: nei nodi intermedi (switch, router) sono implementati solo 2–3 livelli (livelli "media"), mentre solo i sistemi finali implementano l'intero stack fino all'applicazione.

Incapsulamento

Definition

Incapsulamento

Processo top-down in cui ogni livello del mittente aggiunge al messaggio (payload) un campo proprio del livello, come **header** (`H`) o **trailer** (`T`); la **decapsulazione** (bottom-up) rimuole e interpreta quei campi negli stessi livelli del ricevente.

Critiche e confronto TCP/IP vs OSI

Info

Critiche al modello OSI

Bad timing (i protocolli TCP/IP concorrenti erano già in uso), **bad design** (modello e protocolli difettosi), **bad implementations** (implementazioni iniziali enormi e lente), **bad politics** (visto come creatura dei ministeri telecom europei).

Critiche al modello TCP/IP:

- Non distingue chiaramente i concetti di **servizi, interfacce e protocolli**.
- Non è generale: poco adatto a descrivere altri stack di protocolli.
- Il **link layer** non è un vero livello; non distingue il livello fisico dal data link.
- Le altre implementazioni di protocolli erano distribuite gratuitamente.

Confronto:

- **ISO/OSI** è *de iure* lo stack standard per le reti di calcolatori, ma è complesso e dettagliato.
- **TCP/IP e UDP/IP** (*Internet Protocol Suite*) sono *de facto* lo stack realmente usato su Internet e nelle reti locali: **TCP** (*Transmission Control Protocol*), **UDP** (*User Datagram Protocol*), **IP** (*Internet Protocol*).

Esistono altri modelli di rete per la comunicazione tra dispositivi, dipendenti dalla situazione, che però restano basati sul modello a livelli.

Parte II

Application Layer

Capitolo 3

Application Layer

Applicazioni di rete

Definition

Applicazione di rete

Insieme di più programmi che girano su sistemi finali diversi (host) e comunicano tra loro tramite la rete.

Le **applicazioni** sono i programmi in esecuzione sui dispositivi che permettono all'utente di accedere ai **servizi**: social network, Web, messaggistica, e-mail, giochi multiutente, streaming video, file sharing P2P, voice over IP, videoconferenza, motori di ricerca, remote login.

Creare un'applicazione di rete significa scrivere programmi che:

- **Girano su sistemi finali diversi**, anche con linguaggi o sistemi operativi diversi.
- **Comunicano sulla rete** (tipicamente via socket) mediante un protocollo specifico.

Non serve scrivere software per i dispositivi del nucleo della rete: i network device **non eseguono applicazioni utente**, e esistono **librerie specifiche** (es. socket) che implementano le funzionalità di rete.

Architetture delle applicazioni

Definition

Architettura dell'applicazione

Struttura dell'applicazione sui vari sistemi finali, decisa dallo sviluppatore al momento della progettazione.

Le due architetture fondamentali sono:

- **Client-server**: nodi eterogenei; un host sempre attivo (il **server**) fornisce servizi richiesti da molti altri host (i client).
- **Peer-to-peer (P2P)**: nodi idealmente omogenei; l'applicazione sfrutta la comunicazione **diretta** tra coppie di host connessi intermittentemente (peer).

Nel client-server valgono le seguenti proprietà:

- I client **non comunicano direttamente** tra loro.
- Il server ha un **indirizzo fisso e noto** (hostname o IP), così un client può sempre contattarlo inviando un pacchetto di richiesta.
- Spesso sono coinvolti **più server** per reggere il carico; il client in genere **non se ne accorge** e li percepisce come un unico server.
- I server multipli possono essere raggruppati in **data center** (grande numero di server in una sede, da alimentare, mantenere e connettere), sparsi come **server distribuiti** nel mondo (da interconnettere e coordinare) o organizzati in **data center distribuiti**.

Nel peer-to-peer valgono le seguenti proprietà:

- Usato per applicazioni **traffic-intensive**; non esiste un server singolo con indirizzo fisso: ogni

client ha il proprio indirizzo.
- Le applicazioni P2P pure sono rare: la maggior parte è **ibrida**, combinando client-server e P2P (es. server che tracciano gli indirizzi degli utenti, con messaggi scambiati direttamente tra host).
- P2P è **scalabile e distribuito**: ogni peer genera carico ma aggiunge anche capacità di servizio distribuendo file agli altri peer.
- P2P è **economico** (niente infrastrutture o banda significativa) ma ha problemi di **sicurezza, prestazioni e affidabilità**.

Comunicazione tra processi

In un'applicazione di rete i **processi** comunicano attraverso la rete, anche su sistemi operativi diversi; tra una coppia di processi comunicanti si identifica tipicamente un **processo client** e un **processo server**, e nelle applicazioni P2P i processi possono **cambiare ruolo**.

Definition

Socket

Interfaccia software tra livello applicazione e livello trasporto: è la "cassetta delle lettere" tramite cui un processo invia e riceve messaggi, assumendo che lato trasporto esista un'infrastruttura che recapita il messaggio alla socket del processo destinatario.

- Le socket costituiscono l'**API** (*Application Programming Interface*) tra applicazione e rete.
- Lo sviluppatore controlla tutto lato applicazione, ma ha **poco controllo** del lato trasporto: solo la **scelta del protocollo di trasporto** (TCP/UDP) e pochi parametri (dimensione massima del buffer, maximum segment size, ecc.).
- Scelto il protocollo, l'applicazione assume come dati i servizi di trasporto che quel protocollo fornisce.

Per recapitare un messaggio servono due informazioni, dato che più applicazioni girano su uno stesso host:

1. Un **indirizzo** dell'host (per trovare l'host giusto in rete).
2. Un **identificatore** del processo destinatario (per trovare il processo giusto nell'host).

L'indirizzo lavora a livello **rete** (routing), l'identificatore a livello **trasporto**:

- In Internet l'host è identificato da un **indirizzo IP** (quantità a 32 bit).
- Il processo è identificato da un **numero di porta**; le applicazioni popolari hanno porte convenzionali (es. Web server su porta 80, mail server con SMTP su porta 25); la lista dei numeri di porta noti è mantenuta da IANA.

Servizi di trasporto

Definition

Quality of Service

Insieme di proprietà che il livello trasporto può implementare su richiesta dell'applicazione.

- **Affidabilità** del trasferimento: i dati inviati da un capo giungono corretti e completi all'altro capo.
- **Throughput**: ritmo a cui il processo mittente può consegnare bit al ricevente (bit/s).
- **Timing**: ogni bit immesso nella socket arriva alla socket del ricevente entro un intervallo di tempo.
- **Sicurezza**: cifratura e decifratura dei messaggi; teoricamente è un problema di trasporto, ma

i protocolli sicuri (es. **TLS**) sono costruiti su TCP, quindi in pratica sta a livello applicazione.

I due protocolli di trasporto si collocano così:

- **TCP** (*Transmission Control Protocol*): prevede un **handshake** tra i processi e un servizio di trasferimento affidabile; per questo è detto **connection-oriented**.
- **UDP** (*User Datagram Protocol*): **leggero**, con servizi minimi; nessun handshake, nessuna garanzia di consegna; per questo è detto **connectionless**.

Protocolli applicativi

Definition

Protocollo applicativo

Insieme di regole che definisce come i processi di un'applicazione, su sistemi finali diversi, si scambiano messaggi.

Un protocollo applicativo definisce quattro cose:

1. I **tipi** di messaggi scambiati (es. richieste e risposte).
2. La **sintassi** dei tipi di messaggio: campi presenti e loro delimitazione.
3. La **semantica** dei campi, cioè il significato dell'informazione contenuta.
4. Le **regole** che stabiliscono quando e come un processo invia e risponde ai messaggi.

I protocolli applicativi più comuni sono i seguenti.

Applicazione	Descrizione
`DHCP`	Dynamic Host Configuration Protocol, assegna indirizzi IP
`DNS`	Domain Name System, traduce nomi di siti in indirizzi IP
`HTTP/HTTPS`	HyperText Transfer Protocol (Secure), trasferisce pagine web
`SMTP/SMTPS`	Simple Mail Transfer Protocol (Secure), invia messaggi e-mail
`SNMP`	Simple Network Management Protocol, gestisce dispositivi di rete
`Telnet/SSH`	Teletype Network (Secure SHell), interfaccia a riga di comando con host remoti
`FTP/FTPS`	File Transfer Protocol (Secure), usato per trasferire file

Alcune applicazioni scambiano **informazioni sensibili** (credenziali, dati personali): sulle reti pubbliche i messaggi vanno protetti, da qui le varianti *secure* dei protocolli.

FTP e FTPS

Definition

FTP

File Transfer Protocol: uno dei protocolli più antichi di Internet (prima versione 1971), usato per trasferire file tra host tramite rete.

- Nella versione base il trasferimento è in **clear-text** (username, password e file leggibili), quindi è adatto ad applicazioni locali o private.
- La versione sicura **FTPS** protegge username e password e cifra i contenuti.
- FTP e FTPS hanno due componenti: il **protocollo** (comandi per mostrare, scaricare, eliminare file, ecc.) e l'**applicazione software** che lo implementa (lato client e lato server, es. `ftp` e `vsftpd` su Linux).

Comandi FTP più comuni (client connesso al server):

- `help`: elenca i comandi disponibili.
- `cd` / `lcd`: cambia directory sulla macchina remota / locale.
- `ls` / `pwd`: elenca file e directory / stampa la directory di lavoro remota.
- `mkdir` / `rmdir` / `delete`: crea directory, rimuove directory, elimina file sulla macchina remota.
- `get` / `put`: copia un file dal server al client / dal client al server.
- `exit`: chiude la connessione.

Capitolo 4

Application Layer HTTP

Web e HTTP

Fino ai primi anni '90 Internet era usato principalmente da ricercatori, accademici e studenti (Telnet per il login remoto, FTP per i file, e-mail); il **World Wide Web**, nei primi anni '90, fu la prima applicazione che catturò il grande pubblico.

Definition

World Wide Web

Insieme di informazione (documenti, immagini, video, audio, ecc.) accessibile su Internet secondo il protocollo **HTTP** (*HyperText Transfer Protocol*).

La comunicazione HTTP è tipicamente **client-server**:

- Un programma client traduce le richieste dell'utente in messaggi HTTP: è il **browser** (Chrome, Firefox, Edge, Safari, ecc.).
- Un programma server esegue le richieste HTTP e restituisce le **risposte** HTTP.

HTTP definisce principalmente due cose:

- La **struttura dei messaggi**.
- **Come client e server si scambiano** tali messaggi.

Il Web e i suoi protocolli sono la piattaforma per applicazioni web-based (YouTube, web-mail, app mobili, social network). HTTP evolve continuamente per bilanciare affidabilità e prestazioni.

URL

Definition

URL

Uniform Resource Locator: stringa che identifica una risorsa (o oggetto) sul Web, con la struttura `[protocol]://[usrinfo@][host][:port][/path][?query][#fragment]`.

I campi del URL:

- **`[protocol]`**: protocollo da usare per accedere alla risorsa (`HTTP`, `HTTPS`, `FTP`, ecc.).
- **`[usrinfo]`** (opzionale): informazioni utente come username e password seguite da `@` (es. `username:password@`); per lo più deprecato per ragioni di sicurezza.
- **`[host]`**: nome o indirizzo IP del server.
- **`[port]`** (opzionale): porta da usare, spesso dedotta dal protocollo.
- **`[path]`**: percorso della risorsa nel server (es. `/path/to/resource`).
- **`[query]`**: preceduta da `?`, specifica possibili richieste.
- **`[fragment]`**: preceduta da `#`, identifica un elemento nella risorsa (es. un punto di un formulario).

Pagine e oggetti:

- Le pagine Web consistono di un **file HTML** (*HyperText Markup Language*) di base e riferimenti a oggetti aggiuntivi (immagini, video, Java applet, ecc.).
- Una pagina con testo HTML e cinque immagini JPEG contiene **6 oggetti**: il file base più le 5

immagini.

- La URL `http://www.someSchool.edu/someDepartment/picture1.gif` si legge: `http` protocollo, `www.someSchool.edu` hostname, `/someDepartment/picture1.gif` percorso dell'oggetto.
- Una query reale come `https://en.wikipedia.org/w/index.php?title=SSC_Napoli` si legge: `https` protocollo, `en.wikipedia.org` hostname, `/w/index.php` percorso, `?title=SSC_Napoli` query; equivale alla pagina `https://en.wikipedia.org/wiki/SSC_Napoli`.

Il protocollo HTTP

HTTP definisce come i client Web richiedono oggetti (es. pagine Web) ai server Web e come i server li trasferiscono ai client.

- Quando l'utente chiede un oggetto (es. cliccando un collegamento), il browser invia al server **messaggi di richiesta HTTP** per gli oggetti della pagina.
- Il server riceve le richieste e risponde con **messaggi di risposta** che contengono gli oggetti.

Info

HTTP e livello trasporto

HTTP usa **TCP** come protocollo di trasporto sottostante; dalla versione **HTTP/3** (2022) può usare anche **UDP**, e circa il 30% del traffico HTTP complessivo viaggia su UDP.

- Il client prima **inizializza la connessione** con il server; stabilita la connessione, browser e server comunicano attraverso le rispettive interfacce **socket**: il client invia richieste nella propria socket e riceve risposte dalla propria socket; analogamente il server riceve richieste e invia risposte nella propria socket.
- Storicamente si usava TCP per i suoi servizi, prima di tutto il **trasferimento affidabile**: TCP garantisce che i messaggi arrivino integri a destinazione (se possibile).
- I servizi di TCP possono però rallentare la comunicazione, quindi HTTP/3 usa anche UDP: si stima che sia circa il **30% più veloce**; si affida a **QUIC** (*Quick UDP Internet Connection*), che implementa l'affidabilità sopra UDP.
- Qui si vede il vantaggio dell'architettura a livelli: le applicazioni HTTP **non si preoccupano** di raggiungibilità o perdita di dati, delegando il lavoro ai livelli inferiori, e risultano più semplici (logicamente e computazionalmente).

Stateless

Definition

Stateless

Proprietà delle applicazioni HTTP: il server **non mantiene informazioni** sull'interazione (sequenza di richieste/risposte) con un client specifico.

- Se un client chiede due volte lo stesso oggetto, il server non considera la richiesta ridondante: reinvia l'oggetto, avendo "dimenticato" il passato.
- La semplicità è importante per server che gestiscono molti riferimenti al secondo (in media ~1000 richieste/s; i grandi motori di ricerca ricevono centinaia di migliaia di query al secondo).
- Non tutte le applicazioni sono stateless: diverse sono invece **stateful**.

Connessioni persistenti e non persistenti

In molte applicazioni client e server comunicano per un periodo esteso scambiando più coppie richiesta-risposta (back-to-back, periodiche o intermittenti). Con un protocollo connection-oriented (TCP o QUIC) si hanno due modalità:

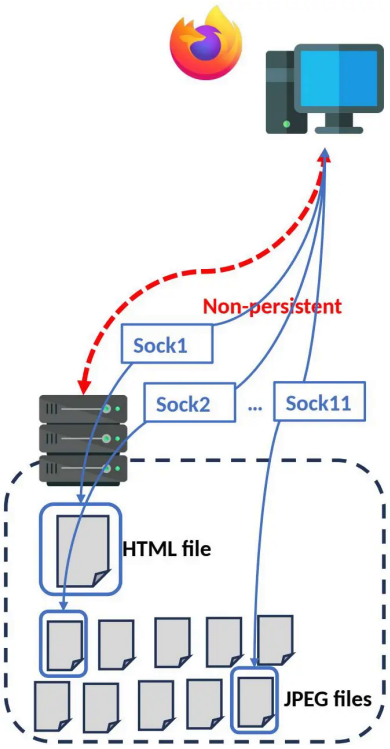
- **Connessione persistente**: tutte le richieste e le relative risposte sono inviate sulla **stessa connessione**.
- **Connessione non persistente**: per ogni coppia richiesta-risposta si stabilisce una **nuova connessione**.

Per impostazione predefinita le applicazioni HTTP odierne usano connessioni persistenti, ma client e server possono essere configurati per la modalità non persistente; le prime versioni di HTTP (1.0) usavano di default la non persistenza.

Connessione non persistente

In una connessione non persistente la connessione si chiude a ogni richiesta servita (es. a ogni oggetto inviato). La sequenza su TCP per una pagina con file HTML base e 10 immagini JPEG (11 oggetti sullo stesso server):

1. Il processo client HTTP inizializza la connessione TCP verso il server sulla porta 80 (porta di default di HTTP): una socket al client e una al server.
2. Il client invia il messaggio di richiesta HTTP con il percorso `/someDepartment/home.index`.
3. Il server riceve la richiesta, recupera l'oggetto (da RAM o disco), lo incapsula in un messaggio di risposta e lo invia tramite la propria socket.
4. Il server chiede a TCP di chiudere la connessione; TCP la termina solo dopo aver certificato che il client ha ricevuto la risposta integra (affidabilità).
5. Il client riceve la risposta, estrae il file HTML e trova i riferimenti alle 10 immagini JPEG.
6. I primi quattro passi si ripetono per ciascuno degli oggetti referenziati.



Example

- Servono **11 connessioni TCP** (e 11 paia di socket) per trasferire l'intera pagina, quindi gli elementi possono arrivare in momenti diversi.
- Le connessioni possono essere stabilite in **sequenza o in parallelo**: ricevuta la pagina base, si possono scaricare in parallelo tutte le immagini; i browser aprono tipicamente 5–10 connessioni TCP parallele, ciascuna per una transazione richiesta-risposta.
- Il parallelismo **accorcia il tempo di risposta** ma **aumenta il costo computazionale**, e stabilire più connessioni TCP può introdurre comunque un overhead significativo.
- HTTP definisce solo il protocollo di comunicazione tra programma client e programma server, **non come i contenuti sono visualizzati**: la resa dipende dal browser (due

browser possono interpretare la stessa pagina in modo diverso).

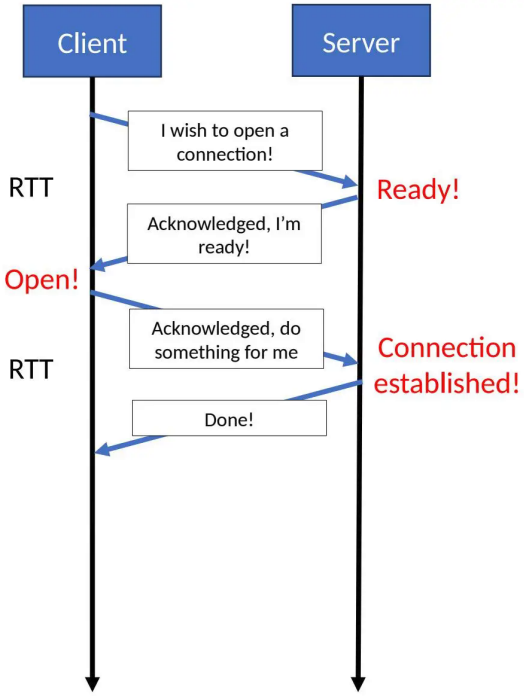
RTT e tempi di risposta

Definition

RTT

Round-Trip Time: tempo che un pacchetto piccolo impiega per andare dal client al server e tornare al client.

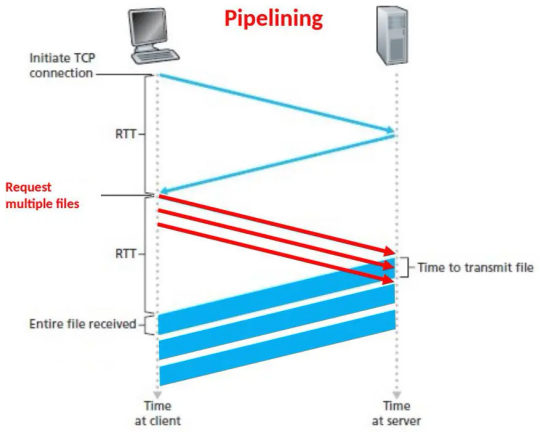
- Per stabilire una connessione TCP si esegue il **three-way handshake**: il client invia un segmento TCP per richiedere la connessione; il server risponde con un segmento di acknowledgement; il client conferma al server.
- Dalla prospettiva dell'RTT servono **2 RTT** per stabilire la connessione (senza perdite); un handshake simile avviene anche alla chiusura.
- Le prime due parti del handshake consumano 1 RTT; il client invia la richiesta HTTP **insieme alla terza parte** del handshake (l'acknowledgment); arrivata la richiesta, il server invia il file HTML, consumando un altro RTT.
- Il **tempo totale di risposta** è circa **2 RTT più il tempo di trasmissione** del file HTML da parte del server.



Connessione persistente

- Il server **lascia aperta** la connessione TCP dopo aver inviato una risposta.
- Un'intera pagina Web (file HTML e oggetti referenziati) e anche più pagine dello stesso server verso lo stesso client viaggiano su **una sola** connessione persistente.
- Il risparmio è di circa **2–4 RTT per oggetto**.

Pipelining: le richieste degli oggetti possono essere fatte back-to-back, senza attendere le risposte; da HTTP/2 richieste e risposte multiple possono essere **interleave** sulla stessa connessione, con meccanismi per **prioritizzare** richieste e risposte. Connessione persistente con pipelining è la modalità di default di HTTP.



Confronto persistente vs non persistente

Persistente:

- Più **veloce**, soprattutto con pipelining.
- Richiede **meno risorse** (soprattutto memoria).
- Più **complessa** da implementare, soprattutto con pipelining.
- La connessione può restare aperta anche inutilizzata: il server la chiude dopo un **timeout** di inattività.

Non persistente:

- **Facile** da implementare e coerente con la *statelessness* di HTTP.
- Richiede più risorse: per ogni connessione si allocano buffer TCP e variabili TCP sia al client sia al server.
- Più **lenta**: 1–2 RTT aggiuntivi per richiesta.
- Le connessioni non possono restare aperte.

Capitolo 5

HTTP messaggi cookie e web caching

HTTP riepilogo

HyperText Transfer Protocol (HTTP): uno dei protocolli più usati su Internet (applicazioni web-based), con le seguenti proprietà:

- È **client-server** e definisce la struttura dei messaggi e come client e server li scambiano.
- **Non** decide come implementare le applicazioni client e server.
- Usa messaggi di tipo **richiesta/risposta**, scritti in **ASCII** leggibile.
- È **stateless**: client e server non ricordano le interazioni passate.
- Assume un trasporto affidabile e connection-oriented (TCP, recentemente anche UDP).
- Usa connessioni persistenti e non persistenti.

Formato dei messaggi

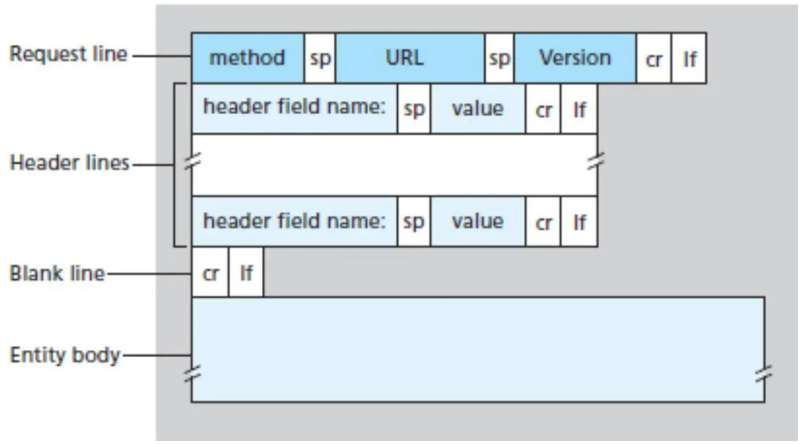
HTTP ha due formati distinti:

- **Richiesta**: specifica il comando (**metodo**) che il server deve eseguire (es. dammi una pagina, elabora un form).
- **Risposta**: riporta l'esito del comando (successo, fallimento, ecc.) e i possibili dati (es. il file richiesto).

Messaggio di richiesta

La richiesta include i seguenti campi:

- **Method**: comando richiesto al server.
- **URL**: identifica l'oggetto su cui operare.
- **Version**: versione HTTP (es. `HTTP/1.1`).
- **Header lines**: parametri della richiesta, in numero e tipo non fissi; ogni riga contiene nome e valore del parametro (es. persistenza della connessione; sono possibili header personalizzati).
- **Body**: dipende dal metodo, contiene dati potenzialmente associati al comando (es. testo di un form).



I campi sono separati da caratteri speciali: **sp** (spazio), **cr** (carriage return, `\r`), **lf** (line feed, `\n`).

Metodi principali:

- **`GET`**: recupera oggetti (risorse) dal server; è il comando più usato, poiché ogni pagina richiesta va recuperata; il body è vuoto.
- **`POST`**: imposta informazioni dentro oggetti (risorse) del server; il body contiene le informazioni da inviare. Le richieste generate da un form non usano necessariamente `POST`: i form HTML usano spesso `GET`, includendo i dati nell'URL richiesto.
- **`HEAD`**: come `GET`, ma il server risponde senza restituire l'oggetto; usato spesso per il debugging.
- **`PUT`**: carica un oggetto su un percorso (directory) specifico di un server Web; usato con strumenti di Web publishing e da applicazioni che caricano oggetti.
- **`DELETE`**: elimina un oggetto su un server Web.

Esempio di richiesta (browser Mozilla/5.0 che chiede `/somedir/page.html` a `www.someschool.edu`):

```HTTP
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

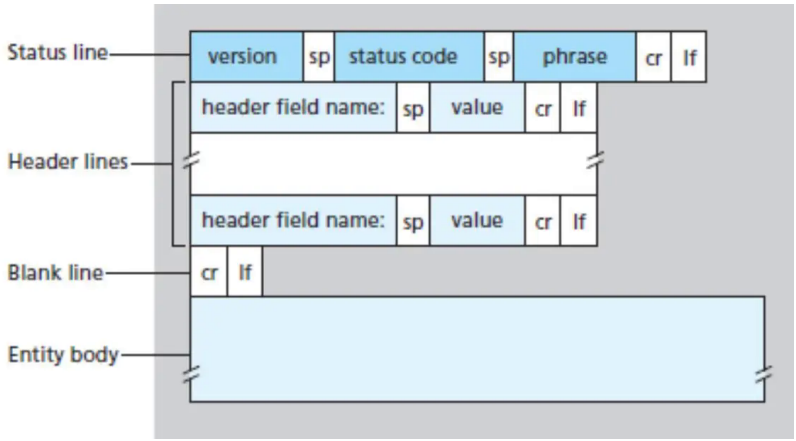
Significato delle righe di header:

- **`Host`**: host su cui risiede l'oggetto; la destinazione non è sempre l'host successivo (il messaggio può passare da un proxy), quindi l'indirizzo nel TCP può differire dall'host reale di destinazione.
- **`Connection: close`**: il browser chiede una connessione **non persistente** (chiusura al termine).
- **`User-agent`**: tipo di browser; utile perché il server può inviare versioni diverse dello stesso oggetto (stesso URL) in base al tipo.
- **`Accept-language`**: preferenza linguistica dell'utente (es. versione francese dell'oggetto, se esiste).

Messaggio di risposta

Il formato è simile alla richiesta, ma al posto della request line c'è una **status line** con:

- **Version**: versione HTTP della risposta del server.
- **Status code**: codice (numero) che specifica l'esito del comando.
- **Phrase**: frase con il risultato della richiesta.



Classi di status code:

- **100–199** informational: informazioni sulla richiesta.
- **200–299** successful: richiesta eseguita con successo.
- **300–399** redirection: servono azioni aggiuntive del client.
- **400–499** client-error: la richiesta non può essere eseguita per un problema del client.
- **500–599** server-error: la richiesta non può essere eseguita per un problema del server.

Example

Codici tipici:

- **200 OK**: richiesta riuscita, l'informazione è nella risposta.
- **301 Moved Permanently**: l'oggetto è stato spostato permanentemente; la nuova URL è indicata nell'header `Location` della risposta.
- **400 Bad Request**: errore generico, il server non ha capito la richiesta.
- **404 Not Found**: il documento richiesto non esiste sul server.
- **505 HTTP Version Not Supported**: versione del protocollo non supportata dal server.

Example

Esempio di risposta:

HTTP

```
1 HTTP/1.1 200 OK
2 Connection: close
3 Date: Tue, 18 Aug 2015 15:44:04 GMT
4 Server: Apache/2.2.3 (CentOS)
5 Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
6 Content-Length: 6821
7 Content-Type: text/html
```

Significato degli header:

- **`Connection: close`**: la connessione verrà chiusa dopo questo messaggio (non persistente).
- **`Date`**: data e ora di creazione/invio della risposta da parte del server.
- **`Server`**: software che ha generato il messaggio (es. Apache; analogo a `User-agent`).
- **`Last-Modified`**: data di creazione o ultima modifica dell'oggetto; utile per il caching, poiché le copie in cache possono essere datate.
- **`Content-Length`**: numero di byte dell'oggetto.
- **`Content-Type`**: tipo dell'oggetto (es. `text/html`): il tipo è dichiarato ufficialmente da questo

header e **non** dall'estensione del file.

Cookie

Un server HTTP è tipicamente **stateless**: questo semplifica la progettazione, riduce l'uso di risorse e permette migliaia di connessioni TCP simultanee; ma la pura statelessness è un limite, perché diverse funzioni Web sono **client-specific** (es. carrello, suggerimenti basati sulle preferenze).

Definition

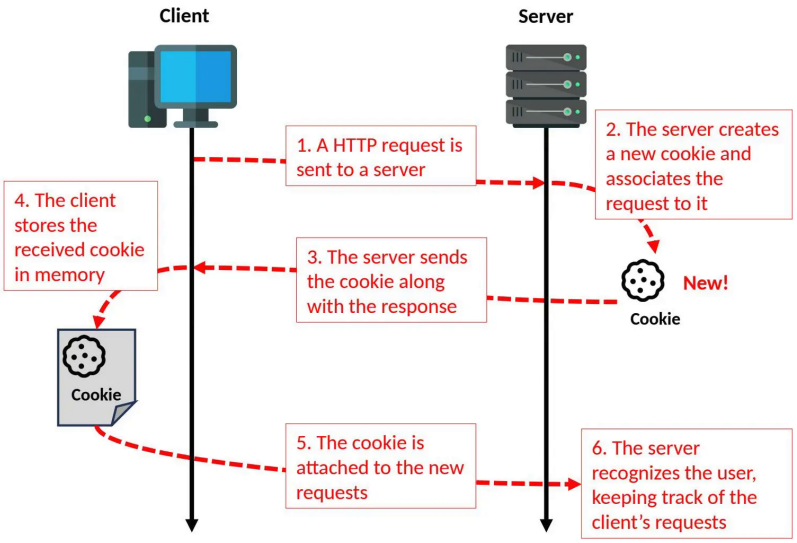
Cookie

Token digitale (ID alfanumerico) creato dal server e consegnato al client, usato dal server per **identificare un client specifico**; i cookie possono avere attributi (es. data di scadenza) e permettono ai siti di tenere traccia degli utenti.

La tecnologia dei cookie ha quattro componenti:

1. Una riga di header `Set-cookie` nel messaggio di risposta HTTP.
2. Una riga di header `Cookie` nel messaggio di richiesta HTTP.
3. Un **file sul sistema client**, gestito dal browser, che associa hostname del server e numero ID del cookie.
4. Un **database sul lato server**, con una voce per ogni cookie.

Funzionamento: alla prima richiesta di un client verso un sito, il server crea un cookie (numero ID) e la relativa voce nel proprio database; la risposta contiene l'header `Set-cookie: 1678`. Il browser aggiunge al file dei cookie hostname e ID; da quel momento ogni richiesta verso quel sito include l'header `Cookie: 1678`, così il server può tracciare l'attività dell'utente nel database (pagine visitate, ordine, tempi).



Usi principali e criticità:

- **Carrello**: il server mantiene la lista degli acquisti durante la navigazione.
- **Registrazione utente**: associando le informazioni dell'utente al cookie nel database (carta di credito, nome, e-mail, indirizzo) per non reinserirle ogni volta.
- **Consigli sui prodotti** basati sulle pagine visitate.

I cookie sono **controversi**: sono considerati un'invasione della privacy, poiché combinando cookie e informazioni fornite dall'utente un sito può sapere molto di lui e potenzialmente vendere tali informazioni a terzi.

Web caching

Definition

Web cache

Entità di rete che soddisfa le richieste HTTP **per conto** di un server Web di origine (*origin server*); le entità intermedie che lavorano per conto di altri sono dette **proxy server**.

- La cache ha un proprio disco dove conserva copie degli oggetti richiesti di recente.
- Il browser può essere configurato per indirizzare **tutte** le richieste HTTP alla cache, che verifica la disponibilità di una copia dell'oggetto.
- Una cache è **sia server** (quando fornisce oggetti) **sia client** (quando li richiede) allo stesso tempo.

Procedura di accesso a un oggetto tramite cache (es. `http://www.someschool.edu/campus.gif`):

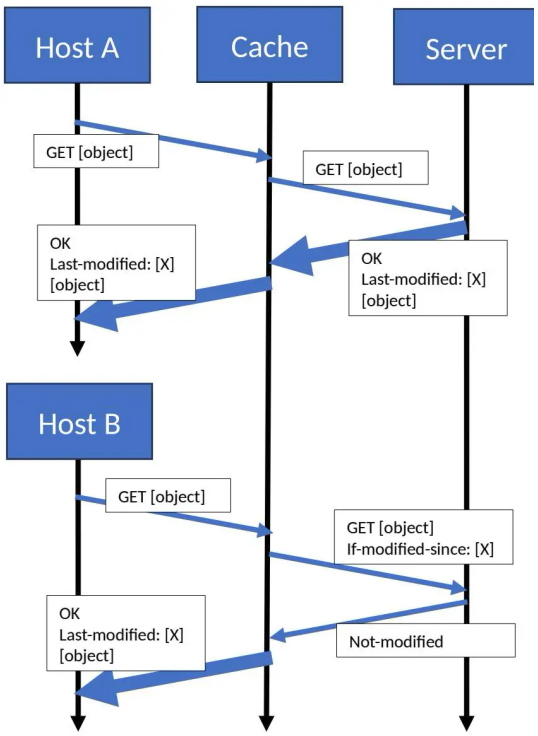
1. Il browser apre una connessione TCP con la cache e le invia la richiesta HTTP dell'oggetto.
2. La cache verifica di avere una copia locale: in caso affermativo la restituisce al browser in un messaggio di risposta.
3. Se non la possiede, la cache apre una connessione TCP con il server di origine e gli invia la richiesta.
4. Ricevuto l'oggetto, la cache ne memorizza una copia e ne invia una al browser in un messaggio di risposta.

Motivazioni del deployment:

1. **Ridurre il tempo di risposta** per il client, specialmente se tra client e cache c'è una connessione ad alta velocità.
2. **Ridurre il traffico** dell'azienda/istituzione verso Internet, riducendo i costi di banda.

Misure e problemi:

- **Hit**: la cache fornisce l'oggetto senza contattare il server di origine; l'**hit rate** (frazione di richieste soddisfatte dalla cache) è tipicamente tra **0.2 e 0.7**, e cresce all'aumentare dei clienti che usano la cache: fino al 70% delle richieste può essere servito localmente.
- **Problema di obsolescence**: la copia in cache può essere datata; per verificarla esiste il **conditional GET**: richiesta HTTP con metodo `GET` e header `If-Modified-Since:`, con cui la cache controlla se l'oggetto è aggiornato e, in tal caso, restituisce la versione memorizzata senza ulteriore comunicazione.
- Anche i **browser** fanno caching locale: memorizzano gli oggetti per non recuperarli dal server; la tecnica migliora molto le prestazioni, ma la versione locale può non essere aggiornata, generando errori piuttosto frequenti.



Oltre il caching

- I proxy servono anche ad **accedere a servizi istituzionali**: le reti aziendali/universitarie forniscono un web proxy istituzionale (es. `proxy.unina.it`).
- Poiché le richieste sono inoltrate dal proxy istituzionale, dal punto di vista del server di origine **tutte le richieste arrivano dall'istituzione**.
- Su Internet esistono diversi proxy commerciali e gratuiti.

Capitolo 6

Application Layer posta e-mail e P2P

Posta elettronica e SMTP

La posta elettronica è una delle applicazioni Internet più antiche e importanti; è tipicamente implementata in stile client-server con due tipi di host:

- **User agent**: applicazione per la gestione della posta da parte dell'utente (es. Pine, K-9 Mail).
- **Mail server**: server che **immagazzina le e-mail** e mantiene le **mailbox** specifiche degli utenti.

Diversamente da HTTP, qui la comunicazione avviene **anche tra server**: servono due protocolli, uno tra user agent e mail server e uno tra mail server e mail server.

Definition

SMTP

Simple Mail Transfer Protocol: principale protocollo di livello applicativo che lavora **tra mail server**; permette ai server di scambiarsi le e-mail, mentre gli user agent **non** usano SMTP.

- Gli utenti non si scambiano posta direttamente (come nella messaggistica istantanea): si preferiscono mail server, più affidabili e specializzati.
- Una volta arrivata sul server, la mail è memorizzata nella mailbox dell'utente, in attesa che l'user agent la scarichi localmente.
- In SMTP esistono un lato client (mittente) e un lato server (ricevente), entrambi sui mail server, che usano il protocollo affidabile **TCP**.

Example

Invio di una mail (Alice → Bob): il processo coinvolge l'agente di Alice, il mail server di Alice, l'agente di Bob e il mail server di Bob.

1. Alice invoca il suo user agent, fornisce l'indirizzo di Bob, compone il messaggio e ordina l'invio.
2. L'agente invia il messaggio al mail server di Alice, dove viene posto in una **coda dei messaggi**.
3. Il lato client SMTP del server di Alice vede il messaggio in coda e apre una connessione TCP verso il server SMTP sul mail server di Bob.
4. Dopo un handshaking iniziale, il client SMTP invia il messaggio nella connessione TCP.
5. Il lato server SMTP del server di Bob riceve il messaggio e lo colloca nella mailbox di Bob.
6. Bob invoca il suo user agent e legge il messaggio quando vuole.

Dettagli del protocollo:

- Il client SMTP stabilisce una connessione TCP alla **porta 25** del server SMTP; se il server è giù, il client **riprova più tardi**.
- Stabilita la connessione, client e server eseguono un handshaking di livello applicativo: **si presentano** prima di trasferire informazioni.
- Durante l'handshaking il client SMTP indica l'indirizzo e-mail del mittente e quello del ricevente.
- SMTP conta sul servizio di trasferimento affidabile di TCP per consegnare il messaggio al server senza errori.

Accesso alla posta

Il deployment di server SMTP è più affidabile dello scambio utente-utente diretto: i server sono sempre accesi e visibili in rete, certificati, e possono ritentare l'invio in continuazione (processo computazionalmente costoso).

Per accedere alle mail sul server serve però un'applicazione client-server aggiuntiva con un protocollo diverso. I più diffusi:

- **POP3** (*Post Office Protocol* version 3).
- **IMAP** (*Internet Mail Access Protocol*).
- **HTTP**.

POP3

Definition

POP3

Protocollo di accesso alla posta estremamente semplice: l'user agent (client) apre una connessione TCP al mail server sulla **porta 110**.

Con la connessione stabilita, POP3 procede in tre fasi:

1. **Authorization**: l'user agent invia username e password per autenticare l'utente.
2. **Transaction**: l'user agent può recuperare i messaggi, marcarli per la cancellazione, rimuovere i segni di cancellazione e ottenere statistiche sulla posta.
3. **Update**: dopo il comando `quit` che chiude la sessione, il server **elimina i messaggi marcati** per la cancellazione.

IMAP

Con POP3 i messaggi possono solo essere scaricati o cancellati: azioni come cercare o organizzare le mail in cartelle non sono previste e vanno fatte sulla macchina locale.

Definition

IMAP

Internet Mail Access Protocol: protocollo di accesso che permette al server di fornire funzionalità aggiuntive rispetto a POP3.

- **Gestione e creazione di cartelle**.
- **Ricerca** nelle cartelle remote di messaggi che rispettano criteri specifici.
- Ottenimento di **solo parti** dei messaggi (es. solo header, solo allegati).
- **Client multipli** connessi contemporaneamente allo stesso server.

Accesso via HTTP

- Sempre più utenti inviano e leggono posta tramite **browser Web**: l'accesso web-based è stato introdotto da Hotmail a metà anni '90 ed è oggi l'approccio dominante (Google, Yahoo!, Virgilio, ecc.), usato da quasi tutte le università e aziende.
- L'user agent è un comune browser che comunica con la mailbox remota via **HTTP** anziché POP3 o IMAP.
- Vale per gli user agent: **i mail server continuano a usare SMTP** per scambiarsi i messaggi.

Peer-to-Peer

Definition

Architettura P2P

Architettura che fa uso minimo (o nullo) di server: coppie di host connessi intermittentemente (**peer**) comunicano **direttamente** tra loro; i peer non sono di proprietà di un fornitore di servizi, ma desktop e laptop controllati dagli utenti.

Il file sharing è l'applicazione naturale:

- Nel client-server il server deve distribuire i file a tutti i client: grave onere per il server, che richiede grande banda.
- Nel P2P i peer che ricevono il file possono **condividerlo a loro volta** con altri peer: in qualche modo i peer sono client e server allo stesso tempo.
- Il P2P scala tipicamente **meglio** del client-server nel file sharing, ma è più complesso da implementare e ha problemi di **sicurezza**, con tutti questi client in comunicazione diretta.

BitTorrent

Definition

BitTorrent

Protocollo P2P popolare per la condivisione/distribuzione di file (utenti stimati 150–170 milioni nel 2023); definisce il comportamento di un **torrent**.

- Un **torrent** è l'insieme dei peer che partecipano alla distribuzione di un file; i peer si scaricano l'un l'altro **chunk** di uguali dimensioni del file (tipicamente 256 kB).
- Un peer che entra nel torrent senza chunk inizia ad accumularne; mentre scarica chunk, **carica** chunk verso gli altri peer.
- Ottenuto il file completo, il peer può lasciare il torrent (egoisticamente) o restare e continuare a caricare (altruisticamente); i peer possono uscire in qualsiasi momento con un sottoinsieme di chunk e rientrare più tardi.

Il tracker

- Ogni torrent ha un nodo di infrastruttura chiamato **tracker**, che tiene traccia dei peer partecipanti (i tracker sono di fatto server).
- Un nuovo peer si **registra** col tracker e lo informa periodicamente del proprio stato; il tracker gli fornisce gli **indirizzi IP** di un sottoinsieme casuale di peer del torrent.
- Il nuovo host tenta connessioni TCP **concorrenti** con tutti i peer della lista (**vicini**); durante l'esecuzione alcuni vicini possono uscire, mentre altri peer (fuori dalla lista iniziale) possono tentare nuove connessioni.
- In ogni istante ogni peer possiede un sottoinsieme di chunk, diverso da peer a peer; **periodicamente** ogni host chiede a ciascun vicino connesso (sulle connessioni TCP) la **lista dei chunk** che possiede.

Strategie del client

Il client in download decide quali chunk richiedere (e a chi) secondo il principio **rarest-first**: hanno priorità i chunk con **meno copie ripetute** tra i vicini; così i chunk più rari vengono ridistribuiti più rapidamente e il numero di copie nel torrent si equalizza.

Il client in upload decide quali richieste servire secondo due principi intrecciati:

- **Trading**: il host da priorità ai **4 migliori vicini** che stanno fornendo dati alla massima velocità; il controllo avviene periodicamente (ogni 10 secondi) e la lista dei 4 migliori si aggiorna di conseguenza.
- **Random selection (optimistic unchoking)**: ogni 30 secondi ogni host sceglie anche **un vicino a caso** e gli invia chunk; se lo scambio casuale risulta buono (i due host sono buoni partner), entrano nelle rispettive liste dei migliori; il processo permette ai peer con velocità di upload compatibili di **trovarsi**.

Capitolo 7

Application Layer DNS

Identificazione degli host

Ci sono due modi per identificare un host:

- **Hostname**: identificatore mnemonico, preferito dalle persone.
- **Indirizzo IP**: identificatore a lunghezza fissa e struttura gerarchica, preferito dai dispositivi di rete.

Definition

DNS

Domain Name System: protocollo di livello applicativo che gestisce la **traduzione da hostname a indirizzo IP**; è un protocollo **client-server** in cui un client DNS chiede a un server DNS una traduzione specifica nome-indirizzo.

- Il DNS è un **database distribuito**, implementato in una gerarchia di server, più il protocollo applicativo che permette agli host di interrogarlo; query e risposte viaggiano in datagrammi **UDP sulla porta 53** (può usare anche TCP porta 53).
- I server DNS sono spesso macchine UNIX con il software **BIND** (*Berkeley Internet Name Domain*), il più diffuso su Linux e sistemi Unix-like.
- È l'unica applicazione che nessuno usa di proposito: lavora per le altre applicazioni e si vede solo quando fallisce.

Un repository centralizzato (tabella unica) è impossibile da gestire per il grande numero di host e per la distanza geografica (e i relativi ritardi): il DNS è quindi un sistema **distribuito e decentralizzato**, ovvero **gerarchico**, **basato su domini** e **implementato tramite un database distribuito**.

- **Singolo punto di guasto**: se crasha, ogni applicazione che risolve un nome si ferma con lui.
- **Traffico**: risponderebbe per ogni richiesta Web e ogni mail inviata sul pianeta.
- **Distanza**: non può essere vicino a tutti: dall'altro capo del mondo ogni lookup paga il viaggio.
- **Manutenzione**: un record per host, con host aggiunti e spostati di continuo.

Il DNS è quindi **distribuito per progetto**: gerarchico, basato su domini, e da nessuna parte completo ; un sistema **distribuito e decentralizzato**, ovvero **gerarchico**, **basato su domini** e **implementato tramite un database distribuito**.

Resolver

Definition

Resolver

Componente usato dalle applicazioni per richiedere la risoluzione dei nomi: costruisce la query DNS e ottiene la risposta, appoggiandosi a un server DNS configurato.

Il name space DNS

- La gerarchia dei nomi è organizzata da **ICANN** (*Internet Corp. for Assigned Names and Numbers*).
- Esistono circa 250 **top-level domain (TLD)**, di due tipi, tutti gestiti da registrar nominati da ICANN:
  - **Generici**: `aero`, `com`, `edu`, ecc.
  - **Geografici** (country): `uk`, `it`, `tv`, ecc.
- Ogni dominio è partizionato in sotto-domini, e così via; lo name space è gerarchico **dalla radice verso il basso**, con parti diverse delegate a organizzazioni diverse.

Alcuni domini generici di riferimento:

Dominio	Uso previsto	Anno	Ristretto?
`com`	Commerciale	1985	No
`edu`	Istituzioni educative	1985	Sì
`gov`	Governo	1985	Sì
`int`	Organizzazioni internazionali	1988	Sì
`mil`	Militare	1985	Sì
`net`	Fornitori di rete	1985	No
`org`	Organizzazioni non-profit	1985	No
`aero`	Trasporto aereo	2001	Sì
`biz`	Imprese	2001	No
`info`	Informativo	2002	No
`name`	Persone	2002	No
`pro`	Professionisti	2002	Sì
`tel`	Dettagli di contatto	2005	Sì
`xxx`	Industria del sesso	2010	No

Regole dei nomi di dominio:

- I nomi sono **percorsi verso l'alto** dal dominio alla radice (es. `eng.mit.edu`); i percorsi **assoluti terminano con un punto**, quelli relativi vanno interpretati.
- È possibile registrarsi sotto **più di un** top-level domain (es. `ibm.com`, `ibm.us`).
- Ogni dominio controlla l'allocazione dei domini sottostanti: creare un nuovo dominio richiede il **permesso del dominio padre**.
- Il naming **non segue** i confini fisici/geografici.
- **Cyber-squatting**: acquisto di domini solo per rivenderli successivamente ad aziende interessate.

Resource record

I record di risorsa (**resource record**) sono le voci del database distribuito: sono ciò che un server restituisce e ciò di cui è fatta una zona. Una risposta DNS trasporta uno o più record, e questi record, sparsi su molti server, **sono** il database, non è mai conservato interamente da nessuna parte.

Definition

Resource record

Voce del database distribuito DNS con cinque campi: **name**, **time to live**, **class**, **type**, **value**. Il tipo decide come leggere gli altri due: cosa significa il nome e cosa significa il valore.

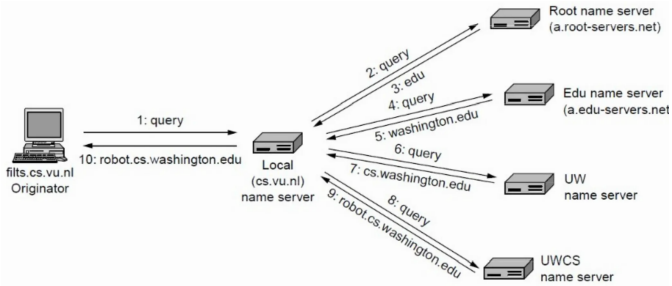
La distribuzione DNS mappa i domini su valori tramite **resource record**, con i seguenti campi:

- **Domain name (NAME)**: dominio a cui il record si applica; è la **chiave di ricerca** principale delle query DNS; per ogni dominio esistono normalmente più record su server diversi.
- **Time to live (TTL)**: vita del record; lungo per host stabili (es. 86400 = 1 giorno), breve per host volatili (es. 60 = 1 minuto); massimo $2^{31} - 1$, circa 68 anni.
- **Class (CLASS)**: sempre `IN` (Internet) per i record Internet; altri codici sono raramente usati.
- **Type (TYPE)**: tipo del record (vedi sotto).
- **Value (RDATA)**: dipende dal tipo; può contenere un dominio, un valore o una stringa ASCII.

Tipi di record principali:

Tipo	Significato	Valore
`SOA`	Start of authority	Parametri della zona, contatti dell'amministratore
`A`	Indirizzo IPv4 dell'host	Intero a 32 bit (più interfacce → più record)
`AAAA`	Indirizzo IPv6 dell'host	Intero a 128 bit
`MX`	Mail exchange	Priorità, dominio che accetta la posta per il dominio
`NS`	Name server	Nome di un server per il dominio (usato nella risoluzione)
`CNAME`	Canonical name	Alias: la lookup prosegue col nuovo nome
`PTR`	Pointer	Alias per il reverse lookup (nome restituito senza procedere)
`SPF`	Sender policy framework	Codifica testuale della policy di invio mail
`SRV`	Service	Host che fornisce il servizio
`TXT`	Text	Testo ASCII descrittivo

Name server e zone



Un unico server DNS centralizzato non avrebbe senso: lo space DNS è partizionato in **zone non sovrapposte**; il partizionamento spetta all'amministratore della zona.

- Ogni zona contiene **uno o più name server**: uno primario e uno (o più) secondario.
- I name server contengono i dati delle porzioni di name space dette zone.

Definition

Zona

Dominio: un nodo dell'albero DNS e tutti i suoi discendenti (gerarchia di naming). **Zona**: la porzione di quel dominio gestita dalla **stessa autorità DNS**; delegando un sottodominio si crea una nuova zona, che però resta parte del dominio padre (es. se `cs.unina.it` è delegata, diventa una zona separata restando parte del dominio `unina.it`). Dominio = gerarchia di naming; zona = autorità amministrativa.

Name resolution

La **risoluzione dei nomi** (name resolution) procede così:

1. L'host interroga un **server DNS locale**.
2. Se il dominio cade sotto la giurisdizione (zona) del server locale, viene restituito un record **authoritative**.
3. Altrimenti il server locale gestisce la query **ricorsivamente** e inoltra query remote: se il server DNS interrogato ha l'indirizzo lo restituisce, altrimenti restituisce il name server di una **zona più bassa**; il processo continua **iterativamente** fino a trovare l'indirizzo.

Query ricorsive e iterative

- **Query ricorsiva**: chi viene interrogato deve produrre la risposta finale, o un errore; chi chiede aspetta e non fa nulla.
- **Query iterativa**: la risposta è o la risposta, o un *referral* al server successivo; chi chiede continua a lavorare.

Suddivisione dei ruoli:

- L'**host** chiede una volta, ricorsivamente: una query fuori, un indirizzo dentro.
- Il **server locale** fa il percorso, iterativamente, un server alla volta (**recursive DNS resolver**).
- I server **root, TLD e authoritative** rispondono normalmente in modo iterativo: restituiscono una risposta autoritativa o un referral, invece di fare ricorsione per il client.

Root server

Sui **root server**:

- Sono il punto di partenza quando non si ha alcuna informazione.
- Sono **13**, da `a.root-servers.net` a `m.root-servers.net`, altamente replicati.

- Sono molto occupati, quindi in genere restituiscono informazioni su zone più basse, non sui singoli record.

Sui tipi di query e record:

- I server DNS locali gestiscono **query ricorsive** (servizio verso i propri host): restituiscono la risposta completa o un errore.
- Molti server DNS (i più occupati, es. quelli vicini alla cima) **non gestiscono** query ricorsive.
- I record sono **authoritative** oppure **cached**: tutte le risposte vengono messe in cache, i record cached **scadono** (ricorda il TTL) e non provengono dal server che gestisce la zona (risposte *non authoritative*); il caching migliora le prestazioni, poiché l'indirizzo può essere restituito subito dal server locale.
- I root server sono interrogati solo in **assenza totale** di informazioni su un dominio; se per `cs.mit.edu` arriva una richiesta e il server ha l'indirizzo del DNS server di `mit.edu`, può interrogare direttamente quello.

DNS caching

Ogni risposta ricevuta da un resolver può essere **messa in cache** e riusata per le query successive:

- La risposta di un resolver ricorsivo è **non-authoritative**: il resolver non è autoritativo per la zona interrogata, che la risposta arrivi dalla cache o sia appena risolta.
- Vengono messi in cache **non solo gli indirizzi** (A/AAAA), ma anche i record **NS** e altre informazioni DNS: questo evita di ripetere l'intero percorso di risoluzione; gli indirizzi dei server TLD in cache fanno sì che una risoluzione di solito **parta sotto la radice**.
- I record in cache valgono solo per il loro **TTL**, dopo il quale vanno rinfrescati.

> *Il caching è ciò che fa funzionare il tutto a questa scala ; e il TTL è ciò che impedisce a una copia in cache di mentire per sempre.*

Un resolver non parte necessariamente dalla radice: **parte dal punto noto più vicino nella gerarchia DNS**.

Example

Se deve risolvere `www.cs.mit.edu` e il record A/AAAA finale non è in cache, usa l'informazione già in cache **più specifica**:

Informazione già in cache	La risoluzione parte da
A/AAAA per `www.cs.mit.edu`	Risposta immediata
NS per `cs.mit.edu`	Server autoritativo di `cs.mit.edu`
NS per `mit.edu`	Server autoritativo di `mit.edu`
NS per `.edu`	Server TLD `.edu`
Nessuna delegazione utile	Root server

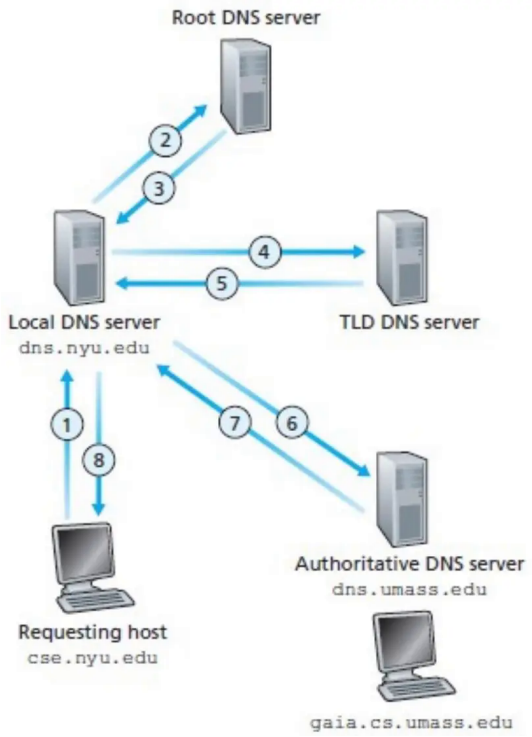
Il resolver locale e i servizi aggiuntivi

Definition

Local DNS server

Server DNS locale (o resolver): tipo di server DNS che **non appartiene strettamente** alla gerarchia, tipicamente gestito dagli ISP; fa da **proxy**, inoltrando le query dell'host alla gerarchia DNS.

- Anche Google fornisce resolver pubblici (Google Public DNS) agli indirizzi `8.8.8.8` e `8.8.4.4`.
- Quando un host si connette, l'ISP gli fornisce gli IP di uno o più server DNS locali, tipicamente "vicini" all'host.



Stub resolver

Un host in genere **non esegue da solo** la risoluzione DNS: sull'host agisce uno **stub resolver**, che riceve le richieste delle applicazioni e invia **una query ricorsiva** al resolver ricorsivo configurato (ISP, istituzione o servizio pubblico); il resolver ricorsivo esegue la risoluzione vera e propria e restituisce la risposta finale. Su molti sistemi Linux è un servizio DNS locale a fare da stub/forwarder: è il caso di `127.0.0.53`.

Servizi aggiuntivi del DNS:

- **Aliasing**: servizio che associa nomi lunghi/complicati (nomi **canonicali**) a nomi più semplici (**alias**); un host con nome complicato può avere alias mnemonici (es. `relay1.west-coast.enterprise.com` → `www.enterprise.com`); l'applicazione può comunque chiedere al DNS il nome canonico di un alias insieme all'indirizzo IP.
- **Load distribution**: distribuzione del carico tra **server replicati** (*round-robin DNS*); i siti trafficati sono replicati su più server con indirizzi IP diversi (più IP associati a un solo nome canonico); il server può **ruotare l'ordine** degli indirizzi nelle risposte, così i client vengono distribuiti sulle repliche.

Strumenti di laboratorio

`nslookup`

Permette di eseguire manualmente la query iterativa normalmente effettuata dal server locale, ed effettuare anche il **reverse lookup** (dal nome all'indirizzo IP, e viceversa). Può anche interrogare uno specifico server DNS, riproducendo a mano il lavoro del resolver ricorsivo.

Lab

```Bash
#Richiesta)
 nslookup www.vatican.va

#Risposta
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
www.vatican.va  canonical name = vatican-g5fcczeabnb9atb6.a03.azurefd.net.
vatican-g5fcczeabnb9atb6.a03.azurefd.net        canonical name = mr-a03.tm-azurefd.net.
Name:   mr-a03.tm-azurefd.net
Address: 150.171.110.52
Name:   mr-a03.tm-azurefd.net
Address: 2603:1061:14:133::1
```

> *Da notare nell'output: la catena di* canonical name *(aliasing in azione) e il fatto che la risposta è* non-authoritative*: la query va al servizio DNS locale `127.0.0.53:53`, che la inoltra al resolver ricorsivo configurato; l'host fa una query ricorsiva, è il resolver a percorrere iterativamente la gerarchia.*

Lab

Verificare la configurazione DNS

```Bash
resolvectl status
# oppure
cat /etc/resolv.conf
```

`dig` (Domain Information Groper)

Oltre a `nslookup`, **`dig`** è uno strumento da riga di comando avanzato per interrogare i server DNS e ispezionare le risposte in un formato molto vicino alla struttura reale dei messaggi DNS.

Funzionalità principali:

- Interrogare **tipi di Resource Record specifici**: A, AAAA, NS, MX, SOA…
- Inviare query a uno **specifico server DNS**.
- Mostrare l'intero **percorso di delega** lungo la gerarchia con il parametro `+trace`.
- Analizzare nel dettaglio **intestazione e flag del messaggio**:
  - `rd` (*recursion desired*), `ra` (*recursion available*);
  - assenza di `aa` → risposta *non-authoritative*;
  - sezione *Question*, record della sezione *Answer* con i relativi TTL;
  - tempo di esecuzione della query, server interrogato, protocollo e dimensione del messaggio.

Filtrare l'output: con i flag `+noall +answer` si isola la sola sezione delle risposte.

Info

Anatomia dell'output di dig vatican.va

L'output rispecchia la struttura del messaggio DNS (header di 12 byte + quattro sezioni):

- **Header**: flag `rd` richiesto e `ra` disponibile; **assenza di `aa`** → risposta *non-authoritative*;
- **Question**: il record richiesto;

- **Answer**: i resource record restituiti, con i relativi **TTL**;
- **Final**: tempo di esecuzione della query, server DNS contattato, protocollo e dimensione del messaggio.

Ricorda la struttura del messaggio DNS: **header** (identificatore copiato nella risposta, così il client può associare risposte a domande, + flag: authoritative answer, recursion desired, recursion available) · **Question** (nome richiesto e tipo) · **Answer** (i record che rispondono) · **Authority** (server che "saprebbero") · **Additional** (record utili, tipicamente gli indirizzi dei server appena nominati).

Lab

Record A con catena di CNAME

Bash

```
)
 dig +noall +answer www.mit.edu A
```

Nella risposta compaiono **due CNAME prima dell'indirizzo**: il nome pubblicato è un alias di un altro nome, e la macchina che infine risponde appartiene a un soggetto completamente diverso. Osserva i **TTL**: l'alias è tenuto per minuti, l'indirizzo finale per pochi secondi.

Lab

Record MX: priorità e mail server

Bash

```
# Record MX: TTL, classe, preferenza (valore più basso = priorità più alta),
    mail server)
 dig +noall +answer mit.edu MX
```

La RDATA di un record MX contiene **due campi**: valore di preferenza e nome del mail server; **valori più bassi = priorità più alta**.

Text

```
mit.edu.   65   IN   MX   100   mit-edu.mail.protection.outlook.com.↑
          ↑    ↑    ↑    ↑     ↑
TTL        classe TYPE pref.  mail server
```

Lab

Record NS: l'NS RRset della zona

Bash

```
# Record NS: server di nome autoritativi per la zona (NS RRset))
 dig +noall +answer mit.edu NS
```

Una query NS può restituire **più record** che insieme formano un **NS RRset**: ciascuno identifica un name server autoritativo per la zona.

Text

```
mit.edu.   1800   IN     NS     asia2.akam.net└────
NAME  └TTL   └CLASS └TYPE └authoritative name server
```

Capitolo 8

Lezione 8 Programmazione di rete

> *Premessa Slide vecchie + audio in attesa delle nuove slide*

Dal livello di applicazione alle socket

La maggior parte delle applicazioni di rete include un programma **lato client** e uno **lato server**: all'esecuzione si creano un processo client e un processo server che comunicano leggendo e scrivendo su **socket**.

- **Protocolli aperti**: protocolli le cui regole sono pubbliche (RFC); chiunque può sviluppare un'applicazione che li parla, e di solito si usano porte specifiche.
- **Protocolli proprietari**: protocolli "custom" di una singola applicazione, non pubblicati; due parti possono comunque comunicare a livello applicativo con un loro linguaggio concordato.

Info

Socket

Elemento centrale della creazione di applicazioni di rete: implementano come scatola nera le funzionalità dei livelli **trasporto** (UDP/TCP) e **rete** (IP); esistono middleware che le incapsulano semplificandone l'uso, ma le API di base restano diffusissime.

I protocolli e i servizi si collocano nel sistema operativo secondo la divisione tra user space e kernel space.

- I protocolli di **livello applicazione** (web, DNS) sono pubblici ma sono implementati da **applicazioni in user space** (per esempio il browser), anche se i loro pacchetti attraversano tutto lo stack.
- Dal livello di **trasporto** in giù tutto è gestito nel **kernel space**, almeno su Unix/Linux e in generale sulle architetture monolitiche.
- Lo schema di interfacciamento è: applicazione (user space) → **API socket** → chiamate di sistema e servizi del kernel → trasporto → rete → accesso; nel kernel il livello di accesso si appoggia ai **driver** delle schede di rete, che realizzano la parte fisica.
- Ogni host implementa gli strati secondo il proprio ruolo: i terminali implementano tutto lo stack, i nodi interni solo i livelli necessari al loro compito (per esempio fino al livello rete per instradare i pacchetti).

Le API di base per socket UDP e TCP sono disponibili in quasi tutti i linguaggi (C, C++, C#, Java, Python, Matlab), con sintassi diverse ma semantica identica.

- Le **Berkeley socket** (dette BSD o POSIX socket) sono state introdotte con la Berkeley Software Distribution (1983), su sistemi già orientati alla comunicazione in rete.
- Sono state poi **standardizzate POSIX** e sono lo standard su Unix, macOS, Windows e altri sistemi; non sono legate all'architettura TCP/IP e gestiscono diversi tipi di canali di comunicazione.
- Sono **native in C**: le implementazioni di altri linguaggi (per esempio Python) si appoggiano a loro volta a un'interfaccia C sottostante, secondo lo schema API di alto livello sopra implementazione a basso livello.

Socket come interfaccia tra processi e trasporto

Il processo si appoggia ai servizi delle socket (API in user space); le API si interfacciano ai servizi del kernel, che si appoggiano allo strato di trasporto configurato sulla socket (TCP o UDP); la comunicazione tra i due host avviene sul trasporto e sopra viaggia il livello di applicazione.

- Il **dominio** di una socket è più generale della rete: oltre alle socket di rete esistono socket locali (`AF_LOCAL`) usabili come comunicazione tra processi sullo stesso host.
- Le socket sono pensate per connessioni **client/server**, ma lo schema non lo impone: entrambe le parti possono mandare messaggi.
- Client e server devono creare socket **dello stesso tipo**: stessa famiglia di indirizzi (non può parlare una in IPv4 e l'altra in IPv6) e stesso tipo di canale di comunicazione.
- Le fasi comuni sono **creazione**, **connessione**, **messaggistica** e **distruzione**: UDP passa direttamente alla messaggistica, TCP prevede la fase di connessione.

Connectionless e connection-oriented

Nel modello **connectionless (UDP)** client e server svolgono questa procedura.

1. Client: crea la socket.
2. Server: crea la socket e la **lega** (`bind`) a porta e indirizzo.
3. Client: invia il messaggio (`sendto`) senza contattare prima il server.
4. Server: legge il messaggio (`recvfrom`), ricavandone IP e porta del client.
5. Server: invia la risposta (`sendto`).
6. Client: legge la risposta (`recvfrom`) e chiude la socket; il server chiude la socket.

Nel modello **connection-oriented (TCP)** la procedura prevede una connessione prima della messaggistica.

1. Client: crea la socket e **si connette** (`connect`) al server.
2. Server: crea la socket, la lega a porta/indirizzo, si mette in **ascolto** (`listen`) e **accetta** (`accept`) le connessioni.
3. Client: invia il messaggio (`send`); server: legge il messaggio (`read`) e risponde (`send`).
4. Client: legge la risposta (`read`); il server chiude la socket **specifica del client** e può tornare ad `accept` per altri client; il client chiude la propria socket.

La funzione socket

`socket(domain, type, protocol)` crea la socket e restituisce un **socket descriptor**, un intero trattato dal sistema come un file descriptor.

- **domain**: tipo di network e famiglia di indirizzi; `AF_INET` (IPv4, il default di internet), `AF_INET6` (IPv6), `AF_LOCAL` (socket locali).
- **type**: semantica del canale di comunicazione; `SOCK_STREAM` (TCP, connection-oriented), `SOCK_DGRAM` (UDP, connectionless), `SOCK_RAW` (gestione manuale del pacchetto, a un livello più basso).
- **protocol**: di solito `0`, cioè il protocollo default per quella combinazione domain/type; si sottospecifica un protocollo particolare solo quando serve.

Info

Everything is a file

Descrittori uniformi: il socket descriptor entra nella tabella dei file aperti del processo, accanto a standard input, output ed error (0, 1, 2); il canale è trattato in modo uniforme rispetto a file, pipe e FIFO, come l'apertura di un canale di comunicazione. `close()` chiude la socket e rimuove

il descrittore dalla tabella dei file aperti.

Strutture degli indirizzi

`sockaddr_in`

La struttura che definisce famiglia, porta e indirizzo di un end point IPv4 è la seguente.

```c
struct sockaddr_in {
    short          sin_family;  // famiglia, tipicamente AF_INET
    unsigned short sin_port;    // porta, in network order (htons)
    struct in_addr sin_addr;    // indirizzo IP
    char           sin_zero[8]; // padding alla dimensione di sockaddr
};
```

- L'indirizzo IPv4 è un intero a **32 bit**, la porta un intero a **16 bit**; le porte basse sono riservate (well-known), le altre sono usate dai processi.
- `sin_zero` è padding di allineamento alla dimensione della struttura generica `sockaddr`.
- La struttura dipende dalla famiglia: per IPv6 si usa `sockaddr_in6`, con indirizzo a 128 bit e dimensione diversa; serve congruenza tra famiglia dichiarata, struttura usata e argomenti delle funzioni.
- `sin_addr` spesso si imposta a `INADDR_ANY`: la socket si aggancia a **qualunque indirizzo** esposto dalle interfacce di rete dell'host; per ascoltare solo da una macchina specifica si indica il suo IP (per esempio `inet_addr("192.168.1.10")`).
- `inet_pton()` converte direttamente un indirizzo in notazione dotted nel formato binario della struttura (funziona per IPv4 e IPv6, purché coerenti con la famiglia); `inet_ntop()` fa l'inverso.

Rappresentazione dei numeri e byte order

Definition

Big endian e little endian

Rappresentazioni degli interi in memoria: big endian pone prima il byte più significativo, little endian prima il meno significativo; host diversi possono usare rappresentazioni diverse, quindi un numero letto con l'ordine sbagliato risulta completamente diverso.

- **Network byte order**: per convenzione i protocolli TCP/IP lavorano in **big endian**; tutti i dati numerici devono viaggiare in rete in questo formato.
- Il programmatore non deve conoscere l'endianess della propria macchina: le funzioni di conversione `htons`/`htonl` (host → network) e `ntohs`/`ntohl` (network → host) fanno la traduzione localmente e rendono il codice portabile; `short` per le porte, `long` per gli indirizzi e gli interi a 32 bit.
- Regola pratica: prima di mandare un numero si converte con `htonl`/`htons`, dopo averlo ricevuto con `ntohl`/`ntohs`; per i byte non serve conversione.

`bind` e assegnazione dell'indirizzo

`bind(socket, address, address_len)` associa la socket a un indirizzo e a una porta locali; restituisce `0` in caso di successo, `-1` altrimenti, e l'ultimo campo serve a verificare la lunghezza attesa della struttura.

- Il bind è **fondamentale per il server**, che deve essere raggiungibile su una porta nota; il client può anche non esporre un indirizzo, perché nel TCP apre lui la connessione e il server risponde sul canale, mentre nell'UDP entrambe le parti devono poter mandare e ricevere.
- Il bind **non crea una connessione**: crea solo l'associazione a un indirizzo; la connessione TCP richiede `listen` e `accept` sul server e `connect` sul client.
- Nessuno garantisce l'assenza di **conflitti di porta**: una porta già occupata rende il bind fallimentare, e quando un canale viene chiuso la porta resta per un periodo non riutilizzabile.

Binding implicito

Definition

Implicit binding

Assegnazione automatica dell'indirizzo e della porta: su una socket non bindata, la prima `sendto` (UDP) o `connect` (TCP) delega al sistema operativo la scelta di un'interfaccia e di una porta libera (solitamente una porta alta), e l'associazione dura fino alla chiusura della socket.

- Un **bind esplicito dopo quello implicito fallisce** (per esempio `EINVAL`): per associare di nuovo l'indirizzo bisogna chiudere la socket e aprirne un'altra.
- `getsockname()` interroga la socket per conoscere indirizzo, porta e interfaccia effettivamente assegnati dal sistema.

UDP

Schema di comunicazione

Nell'UDP ogni invio deve specificare il destinatario e ogni ricezione può restituire il mittente.

- `sendto(osock, obuf, olen, oflags, oaddr, oaddr_len)` invia il datagram; `recvfrom(isock, ibuf, ilen, iflags, iaddr, iaddr_len)` lo riceve e riempie `iaddr` con IP e porta del mittente; entrambe restituiscono i byte effettivamente inviati/ricevuti.
- La trasmissione è **non affidabile**: se un datagram si perde non arriva nessuna notifica; la perdita si può rilevare solo a livello applicativo.
- Entrambe le parti devono essere pronte a ricevere: per ogni `sendto` deve esserci dall'altro lato un processo su una `recvfrom`.
- La configurazione più tipica è **simmetrica**: entrambe le parti fanno il bind, così ciascuna può mandare e ricevere; il client può comunque ometterlo grazie al binding implicito.

Schema essenziale del client (il server è speculare):

C

```c
sockfd = socket(AF_INET, SOCK_DGRAM, 0);
memset(&servaddr, 0, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_port = htons(8080);
```

```
5  servaddr.sin_addr.s_addr = inet_addr("192.168.1.10");
6  sendto(sockfd, hello, strlen(hello), 0,
7         (struct sockaddr *) &servaddr, sizeof(servaddr));
8  n = recvfrom(sockfd, buffer, 1024, 0, NULL, NULL);
9  close(sockfd);
```

- Il buffer di ricezione raccoglie byte: per trattarli come stringa si aggiunge il terminatore alla posizione `n`.
- Il server riceve con `recvfrom` (che riempie `cliaddr` con IP e porta del client) e risponde con `sendto` verso `cliaddr`: può così rispondere anche a un client che non ha fatto bind.

`connect` su socket UDP

Info

Pseudo-canale UDP

La `connect` su una socket datagram **non crea un canale affidabile**: definisce un filtro, scartando i datagram in ingresso non provenienti dal peer con cui ci si è connessi; la comunicazione resta UDP, con perdite comprese.

Flag di `recvfrom` e `sendto`

I flag modificano il comportamento di default (flag a `0`) delle primitive di invio e ricezione.

- **MSG_PEEK**: legge il datagram **senza rimuoverlo** dalla coda di ricezione; consente un'ispezione condizionale (prima guardare cosa c'è, poi estrarre o eventualmente inoltrare a livello applicativo).
- **MSG_TRUNC** (su Linux): accetta il datagram anche quando il buffer di ricezione è più piccolo; la parte fuori dal buffer viene scartata come se fosse stata letta.
- **MSG_WAITALL**: attende l'intero numero di byte richiesto prima di restituire; usato tipicamente con il TCP.
- **MSG_DONTWAIT**: ricezione non bloccante; considera solo ciò che è arrivato in quel momento, senza aspettare l'intero messaggio.

Datagram non attesi

Warning

Datagram senza ricevitore

Un datagram che raggiunge la destinazione ma non trova una socket associata alla porta di destinazione è **scartato dal livello di trasporto**, che può inoltre emettere un messaggio di porta non raggiungibile.

- UDP **non bufferizza i datagram che nessuno attende**: il pacchetto arriva dove deve arrivare, ma a livello di trasporto non c'è nessun processo pronto a prenderlo e viene smaltito.
- Il client lanciato prima del server manda il datagram con `sendto` (che parte comunque) e poi rimane **bloccato sulla ricezione**, oppure riceve l'errore: occorre lanciare prima il server.
- Dev'esserci sempre un **processo in ascolto nel momento giusto**: le sincronizzazioni si ottengono con le primitive bloccanti.

Timeout di ricezione

Per evitare l'attesa indefinita si può impostare un timeout sulla ricezione tramite opzione della socket.

- Si prepara una `struct timeval` (secondi e microsecondi) e la si passa con `setsockopt()` per impostare **SO_RCVTIMEO** sulla ricezione.
- Se il datagram arriva prima della scadenza non succede nulla; alla scadenza la ricezione ritorna `-1` con errore `EAGAIN` o `EWOULDBLOCK` (non equivalenti: si controllano entrambi per compatibilità).
- Il timeout **non ritrasmette**: il datagram che non è arrivato è perso.

Comunicazione su localhost

Nella comunicazione su `localhost` lo stack di rete viene attraversato fino al livello di trasporto, che lo rimanda indietro (**loopback**) con una risalita diretta: nessun pacchetto reale viaggia sulla rete; questi esempi servono solo a prendere confidenza sulla stessa macchina.

Server UDP multi-client

L'UDP permette di servire più client con una sola socket e un solo processo.

- Il server risponde ai datagram con una **singola socket**: riceve, discrimina il mittente (IP e porta) e risponde; non deve stabilire connessioni particolari.
- È più semplice del TCP quando i messaggi sono brevi in entrambe le direzioni e i client sono molti: a ogni pacchetto in arrivo risponde in modo appropriato.
- Non c'è garanzia se due datagram arrivano insieme e il server non è ancora tornato in `recvfrom`: il canale non è affidabile e qualcosa può andare perso.

Buffer delle socket

Ogni socket ha a livello kernel un **send buffer** e un **receive buffer**, gestiti dalle chiamate di sistema; non vanno confusi con il buffer applicativo definito dal programmatore.

- I buffer possono essere **liberi, vuoti o parzialmente occupati**: ciò che si vuole leggere o scrivere può non coincidere con l'intero contenuto.
- Le dimensioni sono configurabili con le opzioni **SO_SNDBUF** e **SO_RCVBUF**, anche in modo asimmetrico tra invio e ricezione; calibrarle significa avere molto controllo sul basso livello.

Invio UDP

Il comportamento dell'invio dipende dalla modalità della socket e dallo stato del send buffer.

- **Comportamento blocking (default)**: `sendto` si blocca finché il datagram non viene accettato; il blocco può essere interrotto (per esempio da un segnale), quindi anche in modalità bloccante non ci si può fidare e i dati vanno controllati.
- **Comportamento non blocking**: `sendto` ritorna immediatamente; se il datagram non può essere accettato viene **scartato**, con `-1` e `EAGAIN`/`EWOULDBLOCK`.
- L'invio è atomico: il datagram va **tutto intero o niente** (niente mezzi pacchetti), anche se il suo payload può essere un frammento del dato di livello applicativo.

Ricezione UDP

Anche la ricezione cambia comportamento tra modalità bloccante e non bloccante, e consuma un datagram per chiamata.

- **Blocking (default)**: `recvfrom` si blocca se non c'è nulla; quando trova qualcosa prende **un intero datagram** per chiamata.
- **Non blocking**: ritorna immediatamente; se la coda è vuota ritorna `-1` (con la relativa giustificazione) e si può gestire con polling (busy waiting).
- **Troncamento**: se il buffer applicativo è più piccolo del datagram, il datagram viene troncato e i byte eccedenti sono persi.

TCP

Connessione e handshake

Definition

Connection-oriented

Canale affidabile di byte-stream tra i due sistemi finali: prima di trasmettere, client e server fanno handshake; sul lato server coesistono una socket in ascolto (welcoming socket) e una socket client-specifica creata dopo l'handshake, su cui avviene lo scambio.

- Il client chiama `connect(socket, address, address_len)`, **bloccante**, verso l'indirizzo del server; restituisce `0` in caso di successo, `-1` in caso di fallimento.
- Il server chiama `listen(socket, backlog)` per mettersi in ascolto, **non bloccante**: dopo la `listen` il codice procede; è l'`accept` il punto di blocco.
- `accept(socket, address, address_len)` è **bloccante**: il processo si ferma lì ad aspettare la connessione e riporta l'indirizzo del client.
- Il **three-way handshake** avviene dentro il livello di trasporto ed è completamente **invisibile** ai programmi: `connect` e `accept` sono il punto di sincronizzazione, da cui la comunicazione prosegue nelle due direzioni.
- `accept` restituisce un **nuovo socket descriptor**: la socket client-specifica insiste sullo stesso indirizzo e porta del bind, mentre la socket di ascolto resta disponibile per le altre connessioni.
- `backlog` è la lunghezza massima della **coda delle richieste pendenti**, unica per tutti i client che si collegano; se la coda è piena, le nuove richieste possono essere rifiutate, ignorate o messe in attesa: decide il kernel, in una zona non garantita dallo standard (di solito sono rifiutate).
- Stabilita la connessione, i due processi comunicano come su un **interprocess communication**: lo stream di byte è utilizzabile con `read`/`write` (come file, pipe e FIFO), oppure con `send`/`recv`, versioni più specializzate per la rete dotate di flag; `sendto`/`recvfrom` restano utilizzabili con il destinatario implicito nella connessione, e `recv` con flag `0` è identica a `read`.
- Chi parla e chi ascolta va definito a livello applicativo: se entrambi si mettono su `send` (o entrambi su `recv`) non comunicano.

Identificazione delle connessioni

Definition

Quadrupla

Identificatore di una connessione TCP: l'insieme (source IP, source port, destination IP, destination port); connessioni con quadruple diverse non sono in conflitto.

- Il server può tenere **più connessioni simultanee sulla stessa coppia IP e porta**: ogni `accept` crea una nuova socket, tutte sullo stesso bind, distinte dalla quadrupla.

- Il binding implicito assegna ai client porte diverse, distinguendo anche client con lo stesso IP.
- I limiti pratici al numero di connessioni sono: i **file descriptor** per processo (limiti del kernel), la **memoria kernel**, le **risorse di CPU**, le **porte disponibili lato client** e altri limiti architetturali.

Chiusura della connessione

La terminazione TCP avviene a **quattro segmenti** (four-way closure), due per ciascuna direzione di chiusura.

- Il client chiama `close`: parte un segmento **FIN**, confermato dall'**ACK** del server; dopo la `close` il client entra nello stato **FIN_WAIT_1** e, ricevuto l'ACK, passa a **FIN_WAIT_2**: la sua socket è chiusa ma il canale resta aperto, in attesa della chiusura del server.
- Quando anche il server chiude la propria socket client-specifica, manda il suo **FIN** (stato **LAST_ACK**); il client risponde con l'ultimo **ACK** e passa allo stato **TIME_WAIT**: non è sicuro che il suo ultimo ACK sia arrivato.
- Il server, ricevuto l'ultimo ACK, può chiudere senza dubbi: l'incertezza sull'arrivo dell'ultimo messaggio resta solo al client, che per questo mantiene la finestra di attesa.

Info

TIME_WAIT

Periodo refrattario dopo la chiusura, con due ragioni: consentire la ritrasmissione dell'ACK finale nel caso in cui il peer, non ricevendolo, ritrasmetta il FIN; lasciar scadere i segmenti ritardati della vecchia connessione, perché non finiscano in una nuova connessione sulla stessa porta.

- Durante il TIME_WAIT la coppia indirizzo/porta non è immediatamente riutilizzabile: chi riapre subito la stessa porta deve attendere la latenza, salvo usare le opzioni sottostanti.
- **SO_REUSEADDR** (opzione di livello socket, POSIX): permette di riutilizzare subito la coppia indirizzo/porta appena chiusa, trascurando il TIME_WAIT; si imposta **prima del bind** e vale sia lato client sia lato server.
- **SO_REUSEPORT** (non standard POSIX, usata per esempio su Linux): permette a **più socket** di fare bind sulla stessa coppia IP e porta e di chiamare `listen` sullo stesso end point; il kernel distribuisce le connessioni in modo bilanciato tra le socket in ascolto, utile per distribuire il carico su sistemi multicore.

Opzioni dei socket

`getsockopt()`/`setsockopt()` leggono e impostano le opzioni di una socket già creata; restituiscono `0` in caso di successo, `-1` altrimenti.

- Parametri: socket descriptor, **livello** dell'opzione (`SOL_SOCKET` per il comportamento della socket, ma anche livelli inferiori come il trasporto TCP o le opzioni IPv4), **nome** dell'opzione, **valore** (impostato o letto) e sua **lunghezza**.
- Le opzioni quindi non riguardano solo il livello della socket: si può scendere fino ai livelli più bassi dello stack di rete.
- Le opzioni vanno impostate **prima del bind**: `SO_REUSEADDR`, `SO_REUSEPORT`, il timeout `SO_RCVTIMEO` e le dimensioni dei buffer `SO_SNDBUF`/`SO_RCVBUF` sono i casi visti.

Flag di `send` e `recv`

I flag specializzano il comportamento di default delle primitive di rete per la singola chiamata.

- **MSG_DONTWAIT**: `send`/`recv` non bloccanti sulla singola chiamata; il controllo diventa attivo (polling).
- **MSG_PEEK**: legge i dati **senza estrarli** dalla coda di ricezione.

- **MSG_WAITALL**: sulla ricezione attende che l'intera lunghezza richiesta sia raggiunta; interrompibile comunque da segnali ed errori, quindi il valore di ritorno va comunque controllato.
- **MSG_NOSIGNAL**: evita che la `send` su canale rotto generi `SIGPIPE`; la schermatura del segnale andrebbe comunque predisposta prima.

Warning

SIGPIPE

Scrivere su un **canale rotto** (l'estremo di lettura ha chiuso) genera il segnale **SIGPIPE**: se non è catturato o ignorato, il comportamento di default è la **terminazione del processo** che lo riceve.

Modalità non bloccante del descrittore

Oltre ai flag per singola chiamata esiste un modo per rendere non bloccante l'intera socket.

- Per la modalità non bloccante dell'intero descrittore non esiste un'opzione standard tra quelle viste: si usa la funzione POSIX `fcntl()`.
- Si leggono i flag con `fcntl(fd, F_GETFL)` e si reimpostano con `fcntl(fd, F_SETFL, flags | O_NONBLOCK)`: i flag sono **maschere di bit**, quindi al valore letto si aggiunge la modalità non bloccante.
- Da quel momento **tutte** le operazioni sul descrittore (`read`, `write`, `send`, `recv`, `connect`) sono non bloccanti; la funzione vale per qualunque file descriptor, non solo per le socket.

Buffer e comportamenti blocking

Scrittura

In base allo spazio nel send buffer le due modalità si comportano così.

- **Blocking (default)**: `write`/`send` attende che **tutti** i byte richiesti siano accettati dal send buffer; normalmente ci riesce, ma un'interruzione (segnale, timeout) può ridurre i byte scritti: il valore di ritorno va sempre controllato.
- **Non blocking**: non aspetta mai lo spazio sufficiente per tutto; scrive i byte che entrano e ritorna quanti ne ha scritti, oppure `-1` con `EAGAIN`/`EWOULDBLOCK` se il buffer è pieno.

Lettura

La lettura ha una semantica diversa dalla scrittura.

- **Blocking (default)**: `read`/`recv` restituisce i byte disponibili in quel momento, anche **uno solo**, senza aspettare che il buffer si riempia; si blocca soltanto quando il buffer di lettura è **vuoto**.
- Per questo è facilissimo che `read` legga **meno** di quanto richiesto: chi vuole tutti i byte deve ciclare.
- **Non blocking**: ritorna subito con i dati disponibili, oppure `-1` con `EAGAIN` se non c'è nulla.
- C'è un'**asimmetria** tra le primitive: la `write` bloccante è "paziente" (aspetta di trasferire tutto), la `read` è "impaziente" (restituisce subito la parte disponibile).
- Una lettura pari a **0 byte** significa che l'altro estremo ha **chiuso la connessione**.

Letture e scritture robuste

Poiché né `read` né `write` garantiscono di coprire tutti i byte richiesti, si definiscono funzioni **robuste** (safe read/write) che ciclicano sul canale fino al completamento.

```c
while (nleft > 0) {
  n = recv(fd, p, nleft, 0);
  if (n == -1 && errno == EINTR) continue; // interrotta: riprova
  if (n == -1) return -1;                  // errore reale
  if (n == 0) break;                       // connessione chiusa
  p += n; nleft -= n;                      // avanza nel buffer
}
```

- Il **puntatore avanza** nel buffer con l'aritmetica dei puntatori: ogni volta che si riceve o invia una quantità di byte, ci si sposta per trattare i rimanenti.
- Un errore `EINTR` (interruzione) fa riprovare; una lettura pari a `0` indica connessione chiusa e si ritorna quanto letto finora; lo schema è identico per la scrittura robusta.

Server concorrenti

Server sequenziale

Un server con un unico processo gestisce **una comunicazione alla volta**: dal `socket` al `bind`, `listen` e `accept`, la socket client-specifica impegna completamente il processo, che solo alla fine torna in ascolto; le altre richieste restano **in coda nel backlog**.

Server multiprocesso

Il server concorrente multiprocesso crea un **nuovo processo per ogni connessione accettata**, così il padre torna subito in ascolto.

- Sequenza: `socket`, `bind`, `listen`, poi ciclo su `accept`; ogni `accept` crea una nuova socket e subito dopo si fa `fork()`.
- Il `fork` restituisce al padre il **PID del figlio** e al figlio **0**: dentro lo stesso codice si distingue il ramo del padre da quello del figlio.
- Padre e figlio nascono **indistinguibili** (a parte il PID) e condividono i descrittori: il figlio chiude la socket in `listen` e gestisce il canale dedicato al client; il padre chiude la socket del client e torna in `accept`.
- Le chiusure sono simmetriche: ciascun processo chiude solo ciò che non usa; la socket in ascolto resta aperta nel padre.

Figli zombie

Warning

Processi zombie

Se il padre non raccoglie l'**exit status** dei figli, questi restano **zombie**: il descrittore resta occupato in attesa della consegna e, su un server pensato per restare in piedi a lungo, si accumula un **memory leak**.

- Rimedio 1: catturare il segnale **SIGCHLD** con un handler che recupera i figli (per esempio con `wait`): alla morte del figlio il padre riceve il segnale e l'handler lo raccoglie.
- Rimedio 2: impostare `SIG_IGN` su `SIGCHLD`, che equivale a mettere i figli in **detach**: il destino del figlio non è più legato al monitoraggio del padre.
- `sigaction()` è il modo moderno per associare un handler a un segnale: una struttura dati (con il campo `sa_handler`) riceve l'handler come puntatore a funzione e viene agganciata al segnale; all'arrivo del segnale il flusso di controllo è interrotto, si esegue l'handler, poi si riprende.

- `signal()` è l'alternativa storica e più semplice: associa direttamente la funzione al segnale.

Server multithread

Il server multithread replica lo schema del fork, ma i **thread condividono lo spazio di indirizzamento**, il che crea un problema sull'`accept`.

- Due `accept` consecutive scriverebbero il nuovo descriptor **nella stessa variabile**, sovrascrivendolo.
- Si salva temporaneamente il nuovo descriptor e lo si copia in **memoria dedicata allocata dinamicamente** (`malloc`), passata come argomento a `pthread_create()`.
- `pthread_create(&tid, attr, routine_avvio, arg)` crea il thread, che lavora nello spazio di indirizzamento del processo principale e gestisce in esclusiva il client.
- La memoria allocata va **deallocata**: la gestione della distruzione può essere centralizzata o delegata al thread in uscita, che sa cosa distruggere.
- Il thread può essere creato o messo in **detach**, per non doverne attendere la terminazione.

Server multiplexing con `select`

Definition

Multiplexing

Tecnica per gestire più client contemporaneamente con un **unico processo**, senza creare thread o processi: `select()` monitora insiemi di file descriptor e si sblocca appena uno di loro è pronto; evita la proliferazione di processi o thread al crescere dei client, al prezzo di overhead e maggiore complessità del codice.

- `select(nfds, readfds, writefds, exceptfds, timeout)` lavora su canali di I/O in generale, non solo socket: rimane bloccata finché un descrittore non è pronto in lettura o scrittura, si verifica un evento, scade il timeout o arriva un'interruzione.
- **nfds** è il range dei descrittori da controllare: si passa il numero massimo di descrittori **più uno**; c'è un limite al numero di descrittori monitorabili.
- I tre insiemi: descrittori su cui aspettare dati **da leggere**, descrittori attesi **a scrivere** e monitoraggio delle **eccezioni** (se non interessa, `NULL`).
- **timeout**: `NULL` per un'attesa indefinita, oppure una `struct timeval` (secondi e microsecondi) per un tempo di attesa definito.
- Valori di ritorno: **positivo** = numero di descrittori pronti; `0` = timeout scaduto; `-1` = errore o interruzione.
- Gli insiemi sono variabili `fd_set` gestite con le macro `FD_ZERO` (azzera), `FD_SET` (aggiunge), `FD_CLR` (toglie), `FD_ISSET` (verifica l'appartenenza).
- La `select` **modifica gli insiemi**: dopo lo sblocco contengono solo i descrittori che si sono resi pronti; l'insieme di lavoro va quindi **reimpostato a ogni iterazione** partendo dall'insieme completo.

C

```c
FD_ZERO(&rset);
FD_SET(sockfd, &rset);
ret = select(sockfd + 1, &rset, NULL, NULL, NULL);
if (ret > 0 && FD_ISSET(sockfd, &rset))
  n = recv(sockfd, buffer, 1024, 0);
```

Le condizioni che svegliano la `select` sono le seguenti.

- **In lettura**: dati nel receive buffer (di default ne basta **un byte**), chiusura del lato di invio del

peer, errore sul canale, richiesta di connessione pendente su una socket in `listen`.
- **In scrittura**: spazio disponibile nel send buffer (anche solo in parte: lo sblocco non garantisce spazio per l'intero messaggio), connessione TCP stabilita (`connect` completata), errore sul canale.
- **In eccezione**: condizioni eccezionali del livello di trasporto, come gli **out-of-band data**, che possono richiedere una gestione rapida dell'evento.

Poiché la `select` si sblocca solo quando c'è qualcosa da fare, l'operazione successiva sul descrittore pronto **non si blocca**: si è certi che ci sia un dato da ricevere (o spazio per scrivere).

Schema del server echo

Lo schema classico combina socket in ascolto e socket dei client in un unico loop.

- Si mantiene un insieme **allset** con tutti i descrittori da monitorare (la socket in `listen` più le socket dei client) e una **copia di lavoro** passata alla `select`, ricostruita a ogni ciclo.
- Un array di client con slot inizializzati a `-1` registra le socket attive; un indice massimo delimita la scansione dell'array.
- Allo sblocco: se è pronta la socket in `listen` si fa subito `accept` (la connessione è già arrivata), si inserisce il nuovo descrittore nel primo slot libero e si aggiornano massimo e range.
- Per ogni client pronto si legge: se la lettura ritorna `0` il client ha chiuso, si rimuove il descrittore dall'insieme e si libera lo slot; altrimenti si scrive la risposta sulla stessa socket.
- Con un unico loop si servono tutti i client, evitando la proliferazione di processi o thread.

Client multiplexing

Un client può essere reso **multiplexed** in un unico processo: la `select` monitora insieme lo **standard input** e la **socket**; se arriva input da tastiera lo si scrive sulla socket, se arrivano dati dalla socket li si stampa su schermo: due canali gestiti contemporaneamente senza thread aggiuntivi.